POLITECNICO DI TORINO

Corso di Laurea in Ingegneria Gestionale



Politecnico
di Torino

Tesi di Laurea

Analisi e Mitigazione delle Vulnerabilità in una Web Application

Relatore

Prof. Andrea Atzeni

Candidato

Alessandra Sidarous Farag

Anno Accademico 2024-2025

“La parola più bella sulle labbra del genere umano è Madre,

e la più bella invocazione è “Madre mia”.

- Khalil Gibran

Sommario

Nel contesto attuale dello sviluppo software, la sicurezza delle applicazioni web costituisce un aspetto cruciale, spesso determinante per l'affidabilità e la resilienza dei sistemi esposti in rete. L'aumento della superficie d'attacco, dovuto alla complessità crescente delle tecnologie impiegate e all'interconnessione continua tra servizi, impone l'adozione di metodologie strutturate per l'individuazione, l'analisi e la mitigazione delle vulnerabilità applicative. Tali metodologie trovano applicazione tanto in ambito professionale quanto in contesti formativi, dove la progettazione sicura assume un ruolo centrale nella preparazione tecnica degli sviluppatori.

In questo quadro si inserisce il progetto illustrato nella presente tesi, incentrato sull'analisi e il rafforzamento della sicurezza di *Twittle*, una web application sviluppata a fini didattici. L'attività si è articolata in tre fasi distinte ma interdipendenti: l'identificazione automatizzata delle vulnerabilità mediante lo strumento OWASP ZAP, l'applicazione di tecniche di mitigazione mirate e, infine, la verifica dell'efficacia degli interventi realizzati attraverso una nuova fase di scansione. L'obiettivo complessivo del lavoro è stato quello di ridurre in modo significativo il rischio associato all'applicazione, validando al contempo un processo metodologico replicabile, basato su strumenti concreti e su una logica iterativa ispirata ai principi del Secure Software Development Lifecycle (SSDLC).

Indice

1 Introduzione **7**

2 Riferimenti Teorici e Strumenti di Analisi **9**

2.1 Secure Software Development Lifecycle (SSDLC). 9

2.2 Vulnerability Assessment. 9

2.3 Tecniche di Analisi del Codice e dell'Applicazione. 10

2.3.1 Analisi Statica (SAST). 10

2.3.2 Analisi Dinamica (DAST). 10

2.3.3 Analisi Interattiva (IAST). 10

2.4 Code Review come tecnica di prevenzione. 10

2.5 OWASP 11

2.5.1 OWASP Code Review Guide 11

2.6 Aree critiche di revisione. 12

2.6.1 Injection. 12

2.6.2 Broken Authentication and Session Management. 12

2.6.3 Cross-Site Scripting. 13

2.6.4 Cross-Site Request Forgery (CSRF). 13

2.6.5 Security Misconfiguration. 14

2.6.6 Sensitive Data Exposure. 15

2.6.7 Insecure Direct Object References (IDOR). 15

2.6.8 Unvalidated Redirects and Forwards. 15

2.7 Strumenti per il rilevamento delle vulnerabilità. 16

2.3.1 SonarQube 16

2.3.2 OWASP Zed Attack Proxy 16

2.3.3 Burp Suite 17

2.3.3 Conclusione 17

3 Valutazione delle vulnerabilità di Twittle **19**

3.1 Panoramica di Twittle 19

3.1.1 Struttura Funzionale del Sito. 19

3.1.2 Aspetti Tecnici e Implementativi. 20

3.1.3 Considerazioni sulla Sicurezza. 21
3.2 Analisi dinamica dell'applicazione Twittle con OWASP ZAP. 21
3.2.1 Configurazione del Proxy e Gestione del Traffico HTTPS. 22
3.2.2 Intercettazione del Traffico HTTPS e Gestione del Certificato CA. 23
3.2.3 Inizio dell'Analisi. 23
3.2.4 Scelta del Criterio di Scansione. 24
3.3 Vulnerabilità Rilevate. 26
3.4 Vulnerabilità ad Alto Rischio (Rosse). 26
3.4.1 Cross Site Scripting (Persistent). 26
3.4.2 Cross Site Scripting (Reflected). 27
3.5 Vulnerabilità a Medio Rischio (Arancioni). 28
3.5.1 Assenza di Token Anti-CSRF. 28
3.5.2 Content Security Policy (CSP). 29
3.5.3 Directory Browsing. 29
3.5.4 Assenza di Anti-clickjacking Header. 30
3.6 Vulnerabilità a Basso Rischio (Gialle). 31
3.6.1 Problematiche di Configurazione dei Cookie. 31
3.6.2 Esposizione di Informazioni Tecniche. 32
3.6.3 Assenza di Strict-Transport-Security Header. 32
3.6.4 Assenza di X-Content-Type-Options Header. 32
3.7 Informazioni (Blu). 33
3.8 Analisi dell'Impatto Complessivo. 34
4 Implementazione delle tecniche di mitigazione 35
4.1 Struttura Logica dell'Applicazione e Controlli di Sicurezza. 35
4.1.1 Sicurezza Mediante Configurazione .htaccess. 36
4.2 Mitigazione delle Vulnerabilità di Tipo Injection. 38
4.2.1 SQL Injection. 38
4.2.2 Logging degli Errori. 39
4.2.3 Content Security Policy (CSP). 40
4.3 Session Management. 41

4.3.1 Sicurezza e Configurazione Iniziale della Sessione. 41
4.3.2 Timeout di Sessione. 42
4.3.3 Gestione dei Cookie. 44
4.4 Mitigazione del Cross-Site Scripting (XSS). 44
4.5 Mitigazione del Cross-Site Request Forgery (CSRF). 47
4.5.1 Implementazione del Synchronizer Token Pattern. 48
4.5.2 Impostazione dell'Attributo SameSite per i Cookie. 48
4.5.3 Verifica delle Intestazioni Origin e Referer. 49
4.6 Mitigazione delle Security Misconfiguration. 50
4.7 Mitigazione del Rischio di Sensitive Data Exposure. 51
4.7.1 Transport Layer Security (TLS). 51
4.7.2 Reindirizzamento HTTPS. 52
4.7.3 HTTP Strict Transport Security (HSTS). 52
4.7.4 Flag Secure e HttpOnly per i Cookie. 52
4.7.5 Prevenzione del Caching di Dati Sensibili. 52
4.7.6 Protezione dei Dati durante l'Archiviazione (at rest). 53
4.8 Mitigazione del Rischio IDOR (Insecure Direct Object Reference). 55
4.8.1 Filtraggio delle Risorse per Utente. 55
4.8.2 Gestione Sicura degli Identificatori Univoci. 55
4.8.3 Considerazioni Pratiche. 56
4.9 Mitigazione dei Redirects e Forwards non Validati. 56
4.9.1 Redirect non validati. 57
4.9.2 Forward non validati. 58
5 Analisi post-mitigazione e Risultati **59**
5.1 Introduzione. 59
5.2 Obiettivi della verifica post-mitigazione. 59
5.3 Metodologia della scansione. 60
5.4 Risultati della Scansione Post-Mitigazione. 60
5.5 Analisi Tecnica delle Vulnerabilità a Basso Rischio (Gialle). 61
5.5.1 Esposizione di Informazioni Tecniche. 61

5.5.2 Assenza Parziale degli Header di Sicurezza HSTS e X-Content-TypeOptions. 62

5.6 Analisi Tecnica delle Informazioni (Blu). 63

5.6.1 Informazioni Potenzialmente Esposte nel Codice e nella Gestione della Sessione. 63

5.6.2 Possibile Vulnerabilità di Cookie Poisoning. 64

5.6.3 Potenziale Controllo Utente su Attributi HTML. 64

5.6.4 Identificazione di Richieste di Autenticazione. 65

5.7 Confronto tra le Scansioni Pre e Post-Mitigazione. 66

5.7.1 Analisi Quantitativa dei Risultati. 66

5.7.2 Analisi Qualitativa degli Interventi di Mitigazione. 66

5.7.3 Vulnerabilità Residuali. 67

5.7.4 Considerazioni Critiche. 68

6 Conclusioni **69**

Ringraziamenti **71**

Bibliografia **72**

Appendice **74**

Capitolo 1
Introduzione

Nel panorama globale contemporaneo, la digitalizzazione permea ogni aspetto della nostra vita quotidiana, dalle interazioni personali alle transazioni commerciali, fino alla gestione delle infrastrutture critiche governative. Tale trasformazione, pur ampliando la connettività e le opportunità di sviluppo, ha inevitabilmente esteso la superficie di attacco, elevando la sicurezza informatica ad una questione di stringente attualità. Nonostante i notevoli avanzamenti tecnologici, i cybercrimini mostrano un aumento costante e preoccupante, con migliaia di dispositivi e servizi attaccati quotidianamente con l'obiettivo di accedere a informazioni sensibili o compromettere il funzionamento di servizi.

Ogni volta che dati personali o aziendali vengono inviati o caricati online, essi diventano intrinsecamente vulnerabili a un vasto spettro di cyberattacchi, i quali possono causare danni significativi a imprese, servizi pubblici e organizzazioni governative. A tal riguardo, il cybercrimine può essere ormai riconosciuto come la terza economia mondiale, generando perdite economiche devastanti [1]. In particolare, "l'Italia è nel mirino", pur rappresentando solo lo 0,7% della popolazione ed l'1,8% del PIL mondiale, nel 2024 ha subito il 10% degli attacchi registrati a livello globale [2].

La cybersecurity, in questo contesto, si occupa di salvaguardare l'integrità, la riservatezza e la disponibilità dei dati attraverso misure di protezione che coinvolgono software, infrastrutture di rete e dispositivi. La necessità di tutelare dati personali, informazioni sensibili e risorse critiche, soprattutto in settori come quello governativo, sanitario o finanziario, rende la sicurezza digitale una priorità strategica a livello globale.

Le applicazioni web, in particolare, hanno visto un'evoluzione rapidissima nell'ultimo decennio, migliorando in modo significativo la qualità e la capacità dei servizi offerti dalla maggior parte delle organizzazioni, divenendo un pilastro fondamentale del business moderno. Esse facilitano interazioni e comunicazioni con i clienti su vasta scala, ma con l'aumento esponenziale del loro utilizzo e della loro complessità, cresce esponenzialmente anche il rischio che soggetti non autorizzati accedano o modifichino dati riservati, compromettendone l'integrità.

Spesso lo sviluppo delle applicazioni web è fortemente orientato all'usabilità e alla ricchezza di funzionalità, trascurando, almeno in fase iniziale, le componenti legate alla sicurezza. Tuttavia, la presenza di vulnerabilità, bug software o errori logici può aprire la strada ad azioni indesiderate, potenzialmente dannose per gli utenti e per l'organizzazione stessa. Piattaforme che trattano dati particolarmente sensibili, come siti di home banking, portali e-commerce o social network, sono costantemente esposte al rischio di violazioni e devono quindi adottare un approccio proattivo, mantenendo il software aggiornato e individuando tempestivamente eventuali punti deboli.

In questo contesto, la presente tesi si propone di analizzare in modo approfondito una serie di vulnerabilità reali presenti all'interno di una web application progettata per finalità didattiche, denominata *Twittle*, una piattaforma di microblogging sviluppata come parte di un progetto accademico.

L'obiettivo è affrontare il tema della sicurezza delle applicazioni web in modo pratico, seguendo un percorso strutturato che parte dall'analisi delle criticità fino alla loro risoluzione. In una prima fase, l'applicazione viene sottoposta a code review e analisi dinamica per individuare le vulnerabilità più comuni secondo le linee guida OWASP. Una volta mappate le problematiche, si passa alla fase di

mitigazione, con l'implementazione di contromisure tecniche. Infine, viene eseguito un secondo ciclo di test per verificare l'efficacia delle soluzioni adottate. Questo permette di valutare il miglioramento ottenuto in termini di sicurezza e di trarre conclusioni concrete sull'approccio seguito.

Capitolo 2
Riferimenti Teorici e Strumenti di Analisi

2.1 Secure Software Development Lifecycle (SSDLC)

Nel contesto dello sviluppo moderno di applicazioni web, la sicurezza non può più essere trattata come una fase isolata da affrontare alla fine del processo di sviluppo. Al contrario, deve essere integrata fin dalle prime fasi di progettazione. È proprio in questa direzione che si muove il *Secure Software Development Lifecycle* (SSDLC), un approccio metodologico che incorpora pratiche di sicurezza in ogni momento del ciclo di vita del software, dalla pianificazione iniziale fino alla manutenzione post-produzione.

Un SSDLC efficace si fonda su alcuni principi chiave. Il primo è il *security by design*, ovvero la progettazione dell'architettura con una mentalità orientata alla sicurezza fin dal principio. Segue la *defense in depth*, un principio che promuove l'adozione di controlli di sicurezza distribuiti su più livelli, in modo da creare una protezione stratificata. Infine, vi è il concetto di *continuous security*, che prevede un monitoraggio costante e un miglioramento continuo della postura di sicurezza dell'applicazione. Questo approccio consente di intervenire precocemente, riducendo sensibilmente i costi e i tempi necessari per correggere eventuali vulnerabilità.

2.2 Vulnerability Assessment

All'interno di un secure SDLC, il *vulnerability assessment* rappresenta il processo sistematico di identificazione, analisi e valutazione delle vulnerabilità di sicurezza presenti nell'applicazione. Questo processo non è limitato a una singola fase del ciclo di sviluppo, ma si articola attraverso diverse attività che variano a seconda del momento di esecuzione e della metodologia utilizzata.

Il vulnerability assessment può essere applicato:

- In fase di sviluppo, con tecniche di prevenzione come l'analisi statica del codice (SAST) e la revisione del codice (*code review*);
- In fase di test o post-deployment, con tecniche di rilevamento come l'analisi dinamica (DAST) e l'analisi interattiva (IAST).

L'obiettivo è quello di garantire una copertura completa delle possibili superfici di attacco, bilanciando interventi preventivi e correttivi.

2.3 Tecniche di Analisi del Codice e dell'Applicazione

Per effettuare un vulnerability assessment efficace, esistono diverse metodologie di analisi che si differenziano per il momento di applicazione e per il tipo di artefatto analizzato [3].

2.3.1 Analisi Statica (SAST)

L'analisi statica (SAST - *Static Application Security Testing*) è una metodologia che esamina il codice sorgente, il bytecode o i binari dell'applicazione senza eseguirla. L'analisi statica opera attraverso diverse tecniche, tra cui la pattern recognition per identificare costrutti di codice noti per essere vulnerabili, la data flow analysis per tracciare il flusso di dati potenzialmente pericolosi dall'input all'output, e la control flow analysis per comprendere i percorsi di esecuzione possibili. Il principale vantaggio dell'analisi statica è la capacità di identificare vulnerabilità potenziali nelle prime fasi del ciclo di sviluppo, quando la correzione è meno costosa. Tuttavia, questa metodologia può produrre falsi positivi e ha difficoltà nell'identificare vulnerabilità che dipendono dal contesto di esecuzione o dalla configurazione dell'ambiente.

2.3.2 Analisi Dinamica (DAST)

L'analisi dinamica (*DAST - Dynamic Application Security Testing*) è una tecnica che testa l'applicazione in esecuzione, simulando attacchi reali per identificare vulnerabilità che si manifestano solo durante il runtime. L'analisi dinamica è particolarmente efficace per rilevare problemi di configurazione, vulnerabilità logiche e comportamenti non previsti che emergono dall'interazione tra componenti diversi. A differenza dell'analisi statica, il DAST non richiede l'accesso al codice sorgente e può identificare vulnerabilità che dipendono dall'ambiente di esecuzione. Tuttavia, la sua efficacia è limitata dalla copertura del codice raggiunta durante i test e generalmente richiede che l'applicazione sia già in uno stato eseguibile.

2.3.3 Analisi Interattiva (IAST)

L'analisi interattiva (*IAST - Interactive Application Security Testing*) rappresenta un approccio ibrido che combina elementi dell'analisi statica e dinamica. L'IAST monitora l'applicazione durante l'esecuzione dei test, tracciando il flusso di dati e identificando vulnerabilità con maggiore precisione e meno falsi positivi rispetto alle tecniche tradizionali.

2.4 Code Review come Tecnica di Prevenzione

Un'altra pratica fondamentale nel contesto del *vulnerability assessment* è la revisione del codice sorgente, ossia l'analisi sistematica del codice per individuare potenziali vulnerabilità prima che vengano sfruttate. La *code review* può essere effettuata sia manualmente sia con l'ausilio di strumenti automatizzati. La revisione manuale, affidata a esperti di sicurezza, è insostituibile per identificare vulnerabilità logiche complesse, errori specifici del contesto applicativo o problemi difficilmente rilevabili con strumenti automatici, come condizioni di gara o errori di business logic. Tuttavia, questa attività richiede tempo e competenze specialistiche, ed è soggetta a variabilità tra revisori.

Gli strumenti automatizzati, invece, applicano regole e algoritmi per individuare pattern di codice vulnerabile in modo rapido e sistematico. Possono essere integrati facilmente nelle pipeline di sviluppo per fornire feedback immediati agli sviluppatori. Tuttavia, possono generare falsi positivi e hanno difficoltà a comprendere il contesto applicativo, limitando la loro capacità di individuare vulnerabilità logiche complesse.

Per questo motivo, la combinazione di revisioni manuali con strumenti automatizzati rappresenta la strategia più efficace, unendo la velocità e la copertura degli strumenti con la profondità e la capacità di giudizio umano.

2.5 OWASP

Nel panorama della sicurezza applicativa, un ruolo centrale è svolto da OWASP (*Open Web Application Security Project*), un'organizzazione no-profit che da anni fornisce risorse e strumenti per aiutare sviluppatori e professionisti della sicurezza a creare software più sicuro. Con la crescente diffusione di servizi online, le applicazioni web sono costantemente esposte ad attacchi informatici, molti dei quali sfruttano errori comuni nel codice sorgente, come input non validati o controlli di accesso carenti.

Tra i contributi più significativi di OWASP vi è la Code Review Guide, un documento di riferimento per chi si occupa di sicurezza applicativa e, in particolare, di revisione del codice [4].

2.5.1 OWASP Code Review Guide

La OWASP Code Review Guide è una risorsa tecnica rivolta a chi si occupa di esaminare il codice sorgente con l'obiettivo di identificarne eventuali vulnerabilità prima che queste possano essere sfruttate in ambiente di produzione. Questa pratica, nota come *secure code review*, è una componente fondamentale di un Secure Software Development Lifecycle (SSDLC), poiché permette di rilevare debolezze logiche o implementative che difficilmente emergono con test automatizzati o dinamici.

La guida non si limita a fornire un semplice elenco di pattern pericolosi, ma propone un approccio metodologico strutturato, che si fonda su alcune fasi essenziali: la comprensione del contesto funzionale e tecnico dell'applicazione; l'analisi dei flussi di dati dall'ingresso all'uscita (*source-to-sink*); l'individuazione dei punti critici nei controlli di sicurezza, come validazione dell'input, gestione delle sessioni, autorizzazione e trattamento dell'output. Inoltre, offre indicazioni concrete per riconoscere e classificare le vulnerabilità più comuni, arricchite da esempi in diversi linguaggi di programmazione.

Uno degli elementi centrali del documento è la classificazione delle aree critiche di revisione, organizzate in dieci capitoli tematici che rappresentano i principali punti deboli da analizzare durante un controllo approfondito del codice. Queste aree coprono sia vulnerabilità tecniche che errori progettuali e logici. Le aree sono le seguenti:

- A1 Injection
- A2 Broken Authentication and Session Management
- A3 Cross-Site Scripting (XSS)
- A4 Insecure Direct Object Reference (IDOR)
- A5 Security Misconfiguration
- A6 Sensitive Data Exposure
- A7 Missing Function Level Access Control
- A8 Cross-Site Request Forgery (CSRF)
- A9 Using Components with Known Vulnerabilities
- A10 Unvalidated Redirects and Forwards

2.6 Aree Critiche di Revisione

Di seguito, verranno esaminate nel dettaglio alcune delle aree critiche trattate nella OWASP Code Review Guide. Per ciascuna, si descriverà la natura delle vulnerabilità associate e si illustrerà come possono essere individuate efficacemente durante l'analisi del codice sorgente.

2.6.1 Injection

Le vulnerabilità di tipo *injection* rappresentano una delle categorie più pericolose e diffuse nel panorama della sicurezza applicativa. Esse si verificano quando un'applicazione accetta input esterni senza una corretta separazione tra dati e comandi, consentendo così all'attaccante di interferire con l'esecuzione delle istruzioni previste. Tali vulnerabilità emergono soprattutto quando i dati forniti dall'utente vengono inseriti direttamente in contesti eseguibili, come ad esempio query SQL, comandi di shell, istruzioni LDAP o espressioni XPath.

La forma più nota è la SQL Injection, che si manifesta quando un'applicazione costruisce dinamicamente una query SQL utilizzando direttamente l'input dell'utente, senza effettuare un'adeguata sanitizzazione. In questi casi, l'input viene interpretato dal database non come dato, ma come codice SQL da eseguire. Questo permette all'attaccante di manipolare le query esistenti, leggere dati sensibili, modificarli o addirittura eseguire comandi amministrativi sul database stesso.

Esempio basilare di codice vulnerabile in PHP:

```
$id = $_GET['id'];
$query = "SELECT * FROM users WHERE id = $id";
```

Un attaccante potrebbe invocare la pagina con un URL del tipo:

```
http://esempio.com/utente.php?id=1 OR 1=1
```

Con questa manipolazione, l'attaccante otterrebbe l'intera tabella degli utenti, eludendo i controlli previsti dalla logica applicativa. In casi più gravi, è possibile accedere, modificare o cancellare dati, e talvolta anche eseguire comandi arbitrari sul server.

Durante una code review, è fondamentale verificare come vengono composte le query e se sono impiegate pratiche sicure come le query parametrizzate che separano codice e dati, annullando il rischio di esecuzione di comandi iniettati.

2.6.2 Broken Authentication and Session Management

La gestione dell'autenticazione e delle sessioni è una componente fondamentale nella sicurezza delle applicazioni web. Quando i meccanismi di login, mantenimento della sessione e controllo dell'identità non sono progettati con attenzione, si aprono potenziali vie d'accesso per attaccanti intenzionati a impersonare altri utenti o ad accedere a risorse riservate.

Queste problematiche possono derivare da pratiche insicure come l'uso di password archiviate in chiaro, la mancanza di rigenerazione dell'ID di sessione dopo il login, o ancora da cookie non configurati in modo adeguato. Anche l'assenza di scadenze per le sessioni inattive o la possibilità di riutilizzare token obsoleti rappresentano segnali di una gestione debole.

Uno scenario tipico potrebbe coinvolgere un'applicazione che mantiene la sessione con un cookie facilmente intercettabile:

```
// Impostazione un cookie di sessione NON sicuro
setcookie("session", $session_id);
```

Senza l'attributo HttpOnly, uno script JavaScript eseguito (es. tramite XSS) potrebbe leggere il cookie:

```
alert(document.cookie);
```

Durante una code review, è essenziale di conseguena analizzare come vengono verificate le credenziali e in che modo il sistema protegge le sessioni attive. Una revisione attenta di questi aspetti consente di individuare vulnerabilità che, se trascurate, possono compromettere l'intero modello di sicurezza dell'applicazione.

2.6.3 Cross-Site Scripting

Il *Cross-Site Scripting*, o XSS, è una vulnerabilità che si presenta quando un'applicazione consente di inserire e visualizzare contenuti controllati dall'utente all'interno di una pagina web, senza adottare adeguate misure per neutralizzare eventuali codici pericolosi. Il risultato è che un attaccante può iniettare script maligni, tipicamente in JavaScript, che verranno poi eseguiti nel browser delle vittime. L'XSS si manifesta in tre forme principali:

- Reflected XSS, in cui il codice malevolo viene riflesso immediatamente nella risposta del server.
- Stored XSS, in cui lo script viene memorizzato, ad esempio nel database, e poi inviato a più utenti in momenti successivi.
- DOM-based XSS, in cui l'iniezione avviene lato client, attraverso manipolazioni del DOM effettuate con JavaScript.

Esempio pratico di Stored XSS:

```
$commento = $_POST['commento'];
echo "<div class='commento'>$commento</div>";
```

Il contenuto del commento viene stampato direttamente nel DOM HTML, senza alcuna validazione o sanificazione. Un attaccante può sfruttare questa vulnerabilità per iniettare JavaScript dannoso, che verrà poi eseguito da chi visita la pagina:

```
<script>
    fetch('https://evil.com/steal?cookie=' + document.cookie);
</script>
```

Nel caso di uno Stored XSS, il codice malevolo inviato dall'attaccante viene salvato nel database insieme ai normali commenti. Quando un utente visita la pagina, lo script viene automaticamente eseguito nel suo browser. Utilizzando `document.cookie`, lo script accede ai cookie dell'utente (se non protetti con `HttpOnly`) e li invia, tramite `fetch()`, a un server esterno controllato dall'attaccante, come `https://evil.com/steal`. Questo consente all'attaccante di rubare la sessione dell'utente senza che questi se ne accorga.

Durante una code review, l'attenzione dovrebbe concentrarsi sui punti in cui il contenuto fornito dall'utente viene visualizzato nel browser, in particolare se ciò avviene tramite `echo`, `print`, o inserimento diretto in elementi HTML o JavaScript. L'assenza di meccanismi di neutralizzazione del contenuto dinamico rappresenta un chiaro campanello d'allarme.

2.6.4 Cross-Site Request Forgery (CSRF)

Il Cross-Site Request Forgery (CSRF) è una vulnerabilità che sfrutta la fiducia di un'applicazione nei confronti dell'utente autenticato. In particolare, CSRF consiste nell'indurre inconsapevolmente l'utente

a inviare una richiesta verso un'applicazione nella quale è già loggato, facendo eseguire un'azione non desiderata come la modifica di dati, l'invio di fondi o il reset della password.

L'attaccante può innescare l'attacco attraverso un semplice link o modulo HTML nascosto, ospitato su un sito malevolo. Poiché il browser include automaticamente i cookie di sessione, la richiesta sembrerà legittima agli occhi dell'applicazione, che la eseguirà come se fosse stata effettuata volontariamente dall'utente.

```
//esempio di CSRF tramite modulo HTML
<form action="https://banca.com/trasferisci.php" method="POST">
    <input type="hidden" name="importo" value="1000">
    <input type="hidden" name="destinatario" value="attacker">
    <input type="submit">
</form>
<script>document.forms[0].submit();</script>
```

Se la pagina di destinazione non verifica l'origine della richiesta né richiede un token CSRF, l'azione verrà eseguita come se fosse legittima.

Durante una revisione attenta del codice, segnali di una possibile vulnerabilità CSRF possono includere la presenza di moduli che non utilizzano token univoci, API esposte senza controllo sull'origine delle richieste, oppure operazioni critiche che possono essere eseguite senza ulteriori conferme o autenticazione aggiuntiva.

2.6.5 Security Misconfiguration

Le vulnerabilità riconducibili a configurazioni di sicurezza errate sono estremamente diffuse e spesso sottovalutate. Esse possono emergere in ogni livello dello stack tecnologico: server, framework, librerie, database, o persino nei file di configurazione del codice stesso. Quando un sistema viene distribuito con impostazioni di default, debug abilitato o directory non protette, può diventare un bersaglio semplice per un attaccante.

Le situazioni più comuni includono directory sensibili accessibili pubblicamente, file di configurazione esposti, moduli amministrativi non protetti, oppure permessi di file e cartelle eccessivamente permissivi. Anche l'inclusione di pacchetti o dipendenze non necessarie può costituire un fattore di rischio.

Dal punto di vista della code review, queste vulnerabilità emergono non solo dall'analisi del codice applicativo, ma anche dall'osservazione di come l'ambiente è configurato e gestito. È quindi importante esaminare anche i file `.env`, script di deploy e configurazioni dei framework utilizzati.

Nello specifico, un errore comune di configurazione è lasciare accessibili file `.env` sul server web. Questi file contengono variabili di ambiente sensibili, come ad esempio:

```
DB_PASSWORD=supersegreta

API_KEY=123abc

SECRET_KEY=xyz456
```

Se il file `.env` non viene protetto correttamente (ad esempio, accessibile da browser tramite `https://example.com/.env`), un attaccante può scaricarlo direttamente e ottenere credenziali critiche, come password del database, token API o chiavi segrete dell'applicazione.

2.6.6 Sensitive Data Exposure

La protezione dei dati sensibili è uno degli aspetti più critici nella sicurezza delle applicazioni web. Con l'aumento della quantità di informazioni personali trattate online, come credenziali di accesso, dati anagrafici, numeri di carte di credito o token di autenticazione, le applicazioni devono garantire che questi dati non siano esposti o trattati in modo inadeguato.

Una vulnerabilità di tipo Sensitive Data Exposure si verifica quando un'applicazione non protegge correttamente informazioni riservate, permettendo a un attaccante di intercettarle, accedervi o persino modificarle. Questo può accadere, ad esempio, se i dati vengono trasmessi su connessioni non cifrate, memorizzati in chiaro nei database o inclusi nei log di sistema.

```
//esempio di password memorizzate in chiaro
$pass = $_POST['password'];
sql_query("INSERT INTO utenti (password) VALUES ('$pass')");
```

Durante un'attività di code review, questi rischi possono emergere attraverso l'analisi dei flussi di dati sensibili, osservando come vengono trattati, memorizzati e protetti lungo tutto il ciclo di vita dell'informazione. Anche un'applicazione solida dal punto di vista funzionale può risultare vulnerabile se i dati più delicati non sono gestiti con attenzione. Per questo motivo, è essenziale che ogni punto in cui tali dati vengono manipolati, sia a livello di codice, sia nella configurazione dell'ambiente, venga valutato con rigore e consapevolezza.

2.6.7 Insecure Direct Object References (IDOR)

L'IDOR, acronimo di *Insecure Direct Object Reference*, è una vulnerabilità logica che si verifica quando un'applicazione permette all'utente di accedere direttamente a risorse interne, file o record di database, semplicemente fornendo un identificatore. Se non vengono implementati controlli adeguati, un utente potrebbe accedere a oggetti che non gli appartengono, semplicemente modificando un parametro nell'URL o nel corpo della richiesta.

Ad esempio, se una URL come `https://esempio.com/documento?id=123` restituisce un file, ma l'applicazione non verifica se l'utente abbia i permessi per visualizzarlo, chiunque conosca o indovini un altro ID potrebbe accedere a dati riservati.

Durante la code review, questa vulnerabilità si manifesta tipicamente nei punti in cui gli identificatori delle risorse (come ID utente, numeri di ordine o percorsi di file) vengono utilizzati direttamente per caricare contenuti, senza essere sottoposti a una verifica di autorizzazione.

2.6.8 Unvalidated Redirects and Forwards

I reindirizzamenti e inoltri non validati si verificano quando un'applicazione consente di indirizzare gli utenti verso URL esterni controllati da input utente, senza effettuare opportune verifiche. Questa vulnerabilità è spesso sottovalutata, ma può avere un impatto significativo, soprattutto quando utilizzata per attacchi di phishing o social engineering.

In fase di revisione del codice, si possono individuare queste vulnerabilità cercando l'uso di funzioni come `header("Location: ...")`, `window.location`, `response.sendRedirect()`, dove la destinazione è costruita direttamente da input utente (`$_GET['url']`, parametri non sanitizzati, ecc.).

Sebbene spesso non comportino compromissioni dirette del sistema, i redirect non validati possono compromettere la fiducia dell'utente e facilitare attacchi più complessi.

2.7 Strumenti per il Rilevamento delle Vulnerabilità

Nel campo della sicurezza delle applicazioni web, disporre di strumenti efficaci per individuare le vulnerabilità è fondamentale per prevenire attacchi e garantire la protezione dei dati. Oltre alla revisione del codice, l'impiego di tool automatizzati consente di analizzare il comportamento dell'applicazione e rilevare errori che potrebbero non essere evidenti in fase di sviluppo.

In questa sezione vengono presentati tre strumenti ampiamente utilizzati in ambito professionale: SonarQube, OWASP ZAP e Burp Suite. Ciascuno di essi adotta approcci differenti per intercettare e segnalare le vulnerabilità, coprendo aspetti complementari del ciclo di vita dello sviluppo software.

2.7.1 SonarQube

SonarQube è una piattaforma open source dedicata all'analisi della qualità del codice, che impiega un approccio di tipo *Static Application Security Testing* (SAST). Pur essendo originariamente orientato al miglioramento della leggibilità e manutenibilità del software, SonarQube ha progressivamente esteso le proprie funzionalità, includendo regole e motori di analisi per la rilevazione di vulnerabilità a livello di codice.

Caratteristiche tecniche [5]:

- Tipologia di test: SAST con supporto per security rules personalizzate
- Modalità di esecuzione: CLI tramite SonarScanner, plugin per IDE, integrazione web-based
- Linguaggi supportati: Java, C#, JavaScript, Python, PHP, C/C++, Go e oltre 25 linguaggi
- Regole di sicurezza: basate su standard OWASP, CWE, SANS Top 25
- Integrazione CI/CD: Plugin disponibili per Jenkins, GitLab CI, GitHub Actions, Azure DevOps
- Licenza: open source con opzioni commerciali

Il rilevamento delle vulnerabilità avviene tramite analisi statica del codice sorgente: il motore di SonarQube scansiona i file alla ricerca di pattern pericolosi, come input non sanificati, comandi eseguiti dinamicamente o la gestione inadeguata delle eccezioni. Ogni problema identificato viene accompagnato da un livello di gravità, una descrizione dettagliata e, ove possibile, una proposta di correzione. I risultati vengono aggregati in una dashboard centralizzata che mostra il livello di sicurezza complessivo del progetto. Successivamente, attraverso i quality gates, SonarQube può bloccare automaticamente il rilascio del software se vengono rilevate vulnerabilità critiche, rendendolo particolarmente adatto per pipeline DevSecOps.

2.7.2 OWASP Zed Attack Proxy

OWASP ZAP (Zed Attack Proxy) è uno strumento gratuito e open source orientato alla *Dynamic Application Security Testing* (DAST). Sviluppato dalla community OWASP, consente di testare applicazioni web in esecuzione, simulando attacchi reali in modo sia automatico che manuale.

Caratteristiche tecniche [6]:

- Tipologia di test: DAST con supporto per test manuali e automatizzati
- Modalità di esecuzione: GUI desktop, modalità headless via CLI, Docker, API REST
- Tipi di scansione: Passive scanning, active scanning, spidering, fuzzing
- Profili di scansione: Policy configurabili per diversi livelli di aggressività e contesti (produzione, staging, sviluppo)

- Integrazione CI/CD: Plugin per Jenkins, script per automation, baseline scan per controlli rapidi
- Licenza: completamente open source

ZAP opera come un proxy intermedio tra client e server, intercettando il traffico HTTP/HTTPS per analizzare in tempo reale le richieste e le risposte scambiate dall'applicazione. Questo approccio consente di rilevare indicatori di vulnerabilità comuni come Cross-Site Scripting (XSS), SQL Injection, o la mancanza di header di sicurezza. Le scansioni possono essere eseguite in modalità passiva, non invasiva, oppure attiva, simulando veri e propri attacchi per testare la reale sfruttabilità delle vulnerabilità identificate.

ZAP è particolarmente apprezzato per la sua semplicità e flessibilità. Permette infatti di combinare test automatici con analisi manuali approfondite, grazie a strumenti integrati come il proxy di intercettazione, il fuzzing (invio di input anomali per testare la robustezza dell'applicazione), e lo spidering, per esplorare automaticamente le pagine e i percorsi accessibili.

Ogni vulnerabilità rilevata viene documentata in un report dettagliato, che specifica l'impatto, la posizione nell'applicazione e fornisce indicazioni sulle contromisure consigliate, facilitando il processo di *remediation*. Queste caratteristiche rendono ZAP uno strumento efficace e versatile per l'analisi dinamica della sicurezza delle applicazioni web.

2.7.3 Burp Suite

Burp Suite è una piattaforma professionale per il testing della sicurezza delle applicazioni web, che combina analisi dinamica (DAST) con funzionalità interattive avanzate (IAST). È ampiamente utilizzata in ambito di penetration testing grazie alla sua completezza e modularità.

Caratteristiche tecniche [7]:

- Tipologia di test: DAST/IAST con estensioni per analisi semi-automatizzate
- Modalità di esecuzione: GUI desktop (Professional/Enterprise), API per automation
- Funzionalità principali: Proxy intercepting, scanner automatico, intruder per attack customizzati, repeater per test manuali
- Estensibilità: BApp Store con oltre 400 plugin, SDK per sviluppo di estensioni personalizzate
- Integrazione CI/CD: Burp Suite Enterprise Edition con API per automation e reporting
- Licenza: versione Community gratuita (limitata), Professional e Enterprise a pagamento

Burp Suite si distingue per il suo proxy integrato, che consente di intercettare, modificare e riprodurre le richieste HTTP/HTTPS in tempo reale. Il suo scanner automatico rileva vulnerabilità come XSS, SQLi, open redirect e accessi non autorizzati, evidenziando le richieste affette e suggerendo exploit possibili. Inoltre, strumenti come Intruder e Repeater consentono all'analista di simulare attacchi mirati, testare l'impatto delle vulnerabilità, e analizzare manualmente i punti critici dell'applicazione. I risultati vengono segnalati in report strutturati, categorizzati secondo gli standard OWASP Top 10, con evidenza visiva dei rischi e raccomandazioni specifiche per la mitigazione.

2.7.4 Conclusione

OWASP ZAP, Burp Suite e SonarQube rappresentano strumenti chiave nella protezione delle applicazioni web, ciascuno con un ruolo specifico nell'individuazione delle vulnerabilità. In particolare, OWASP ZAP e Burp Suite sono utilizzati per test dinamici e interattivi su applicazioni web in esecuzione, mentre SonarQube si focalizza sull'analisi statica del codice, permettendo di rilevare eventuali problemi di sicurezza sin dalle prime fasi dello sviluppo. L'integrazione di questi strumenti

consente di coprire l'intero spettro della sicurezza applicativa, dall'analisi preventiva del codice fino al controllo di applicazioni operative in ambienti reali.

Per maggiore chiarezza, le principali somiglianze e differenze tra queste soluzioni sono riassunte di seguito.

Caratteristica	SonarQube	OWASP ZAP	Burp Suite
Tipo di analisi	SAST (statica)	DAST (dinamica)	DAST + IAST
Fase SDLC	Sviluppo	Test / Produzione	Test manuale / Pentesting
Codice sorgente	Necessario	Non necessario	Non necessario
Esecuzione	CLI, IDE, Web	GUI; CLI, API, Docker	GUI, API (Enterprise)
CI/CD	Integrazione diretta	Script e plugin	Solo con versione Enterprise
Licenza	Open source + commerciale	Open source	Gratuita (limitata) + commerciale

Tabella 2.1: *Confronto tra Strumenti di Rilevamento delle Vulnerabilità*

Nel contesto di questo progetto di tesi, l'analisi di sicurezza delle applicazioni web è stata affrontata attraverso due approcci complementari: la code review e l'analisi dinamica. Questa scelta riflette l'esigenza di combinare un esame approfondito del codice sorgente con una verifica pratica del comportamento dell'applicazione in fase di esecuzione.

Per quanto riguarda l'analisi dinamica, è stato scelto OWASP ZAP come strumento principale. La decisione è stata motivata da diverse considerazioni di carattere tecnico e pratico. Innanzitutto, ZAP è un tool DAST (*Dynamic Application Security Testing*), perciò consente di eseguire test sull'applicazione mentre è in esecuzione, identificando vulnerabilità che non emergono a livello di codice, ma che si manifestano solo durante il runtime, come configurazioni errate, input non gestiti correttamente o problemi di logica applicativa.

Un altro aspetto determinante nella scelta è la gratuità e l'accessibilità del tool: OWASP ZAP è completamente open source e non richiede licenze commerciali, il che lo rende particolarmente adatto per un contesto accademico e per progetti di ricerca. Alla sua accessibilità si affianca una notevole versatilità, grazie alla possibilità di eseguire sia test automatizzati che manuali. Questo consente di calibrare l'attività di analisi in base alle esigenze specifiche del progetto, passando da una scansione rapida a un'ispezione più dettagliata e mirata.

Un ulteriore punto di forza è rappresentato dalla community OWASP, attiva e costantemente aggiornata, che garantisce supporto, documentazione e un allineamento continuo con le evoluzioni del panorama delle minacce.

Le caratteristiche di OWASP ZAP, insieme all'analisi delle vulnerabilità teoriche e agli strumenti presentati in questo capitolo, hanno fornito una solida base per lo studio pratico dell'applicazione *Twittle*, descritta nel capitolo successivo.

Capitolo 3

Valutazione delle Vulnerabilità di Twittle

Nel contesto della sicurezza informatica, i piccoli siti web e le applicazioni legacy sono spesso particolarmente esposti a vulnerabilità note e facilmente sfruttabili, principalmente a causa di una scarsa attenzione dedicata alla sicurezza e di risorse limitate.

Un esempio emblematico riguarda i sistemi basati su CMS (*Content Management System*) open source come WordPress, ancora largamente utilizzati in realtà piccole o amatoriali. Questi sistemi, se non aggiornati regolarmente, possono diventare facilmente preda di attacchi automatizzati che sfruttano vulnerabilità di plugin o configurazioni errate, come dimostrato da numerosi casi riportati da Sucuri, azienda leader nella sicurezza web [8].

In parallelo, OWASP (Open Web Application Security Project), nella sua Top 10 aggiornata al 2021, evidenzia come molte delle minacce più critiche per le applicazioni web restino sorprendentemente costanti nel tempo. Vulnerabilità storiche come l'*Injection* e il *Cross-Site Scripting (XSS)* continuano a occupare le prime posizioni della classifica, a dimostrazione del fatto che non si tratta solo di problemi tecnici, ma anche del risultato di pratiche di sviluppo superficiali. OWASP sottolinea come queste problematiche siano particolarmente frequenti in progetti che non integrano la sicurezza sin dalle prime fasi, dove l'assenza di validazione degli input, controlli deboli sull'autenticazione e logiche di autorizzazione mal implementate favoriscono l'emergere di falle ben note e ampiamente documentate.

3.1 Panoramica di Twitte

L'applicazione web oggetto di questa tesi, denominata *Twittle*, è stata realizzata come progetto accademico per un esame universitario, con lo scopo di consolidare le competenze tecniche nella realizzazione di applicazioni web mediante teconologie standard come PHP, JavaScript, HTML e CSS.

La piattaforma si ispira esplicitamente al funzionamento del noto social network Twitter (oggi X), riproponendone in chiave semplificata le principali funzionalità di comunicazione testuale tra utenti. Twittle si configura come un social network in cui gli utenti possono pubblicare brevi messaggi testuali, detti *twit*, della lunghezza massima di 140 caratteri. Gli utenti registrati possono autenticarsi, visualizzare la propria bacheca personale e inserire nuovi contenuti, mentre i visitatori anonimi possono esplorare i tweet pubblici attraverso la sezione scorpi, accessibile anche senza autenticazione. La struttura del sito è organizzata in moduli distinti ma coerenti fra loro, accessibili tramite un menu di navigazione presente in tutte le pagine e adattato dinamicamente allo stato dell'utente (anonimo o autenticato).

3.1.1 Struttura Funzionale del Sito

Twittle si articola in una serie di moduli principali, ciascuno con una funzione ben definita:

- Home (`home.php`): pagina introduttiva accessibile a tutti, presenta le finalità del sito e offre collegamenti rapidi alle altre sezioni.

- Registrazione (`registrazione.php`): consente la creazione di un nuovo account. L'utente è tenuto a inserire nome, cognome, data di nascita, indirizzo, username e password. La validazione dei dati avviene lato server secondo regole sintattiche rigorose.
- Login (`login.php`): permette l'autenticazione dell'utente tramite inserimento di username e password. È inoltre presente un pulsante che consente di proseguire la navigazione senza autenticazione. Se l'accesso ha successo, l'utente viene reindirizzato alla propria bacheca, in caso contrario, viene visualizzato un messaggio di errore.
- Logout: consente all'utente di terminare la sessione e tornare allo stato anonimo. Il menu si aggiorna di conseguenza, disattivando la voce "Logout" e riattivando quella "Login".
- Bacheca (`bacheca.php`): accessibile solo agli utenti autenticati, presenta l'elenco cronologico dei tweet pubblicati dall'utente. È disponibile un sistema di filtro per intervallo temporale, che consente di selezionare e visualizzare solo i tweet compresi tra due date specifiche. Se l'utente non ha ancora pubblicato contenuti, viene mostrato un messaggio che lo invita a farlo.
- Scrivi (`scrivi.php`): interfaccia per l'inserimento di un nuovo tweet, con limite massimo di 140 caratteri, disponibile solo per utenti autenticati. Una volta inviato il messaggio, l'utente viene reindirizzato alla bacheca.
- Scopri (`scopri.php`): sezione pubblica che permette la visualizzazione di tutti i tweet presenti nella piattaforma, indipendentemente dall'autore. È accessibile anche da utenti anonimi, ed è il punto di arrivo per coloro che scelgono di continuare senza autenticarsi.

Il menu di navigazione, costantemente presente in alto su ogni pagina del sito, fornisce un accesso alle principali sezioni dell'applicazione: *Home*, *Registrati*, *Login*, *Bacheca*, *Scrivi*, *Scopri* e *Logout*. Le voci del menu sono sempre visibili, ma il loro stato (attivo o disabilitato) dipende dal contesto dell'utente, ovvero se è autenticato oppure anonimo. Nello specifico, le voci *Registrati* e *Login* risultano attive solo per utenti anonimi e vengono disabilitate automaticamente al momento dell'autenticazione. Al contrario, le voci *Logout*, *Bacheca* e *Scrivi* diventano cliccabili esclusivamente quando è presente una sessione utente attiva. La pagina *Scopri* è invece sempre accessibile e cliccabile, indipendentemente dallo stato di autenticazione, mentre *Home* adatta dinamicamente i contenuti (es. pulsanti o messaggi) a seconda che l'utente sia autenticato o meno.

Dopo un login riuscito, ogni pagina del sito visualizza, nell'intestazione in alto a sinistra, lo username dell'utente connesso e una preview fino a 30 caratteri dell'ultimo tweet da lui pubblicato, qualora presente. Infine, nel caso un utente anonimo tenti di accedere manualmente, tramite modifica dell'URL, a pagine riservate come *bacheca.php* o *scrivi.php*, il sistema blocca la navigazione e restituisce un messaggio che notifica la mancanza di autorizzazione, informando l'utente della necessità di autenticarsi per usufruire di tali funzionalità.

3.1.2 Aspetti Tecnici e Implementativi

Sul piano tecnico, l'applicazione è realizzata utilizzando PHP per la logica server-side, affiancato da HTML, CSS e JavaScript per la parte di presentazione e interazione lato client. I dati relativi agli utenti e ai messaggi pubblicati (i "twit") vengono gestiti tramite un database relazionale MySQL, con una struttura semplice composta principalmente da due tabelle: una per le informazioni anagrafiche e di autenticazione, l'altra per i contenuti testuali pubblicati dagli utenti.

Il layout grafico del sito è mantenuto coerente grazie all'impiego di un foglio di stile CSS condiviso, mentre per le componenti comuni, come l'intestazione, il menù e il piè di pagina, è stato adottato un sistema di inclusione modulare che consente di mantenere il codice più ordinato e facilmente aggiornabile.

3.1.3 Considerazioni sulla Sicurezza

Nonostante si tratti di un progetto accademico, sono state integrate alcune buone pratiche di base, come l'utilizzo dei *prepared statements* per tutte le operazioni di interazione con il database. Questa scelta contribuisce a prevenire attacchi di tipo SQL injection, che rappresentano una delle vulnerabilità più diffuse nelle applicazioni web prive di protezione.

La comunicazione tra client e server avviene tramite protocollo HTTPS, che sfrutta TLS per garantire la cifratura dei dati in transito. Sebbene il certificato TLS non sia stato configurato manualmente dallo sviluppatore, essendo già disponibile nell'ambiente di lavoro fornito per il progetto, esso consente comunque di stabilire connessioni sicure, proteggendo le informazioni da possibili intercettazioni o attacchi man-in-the-middle.

Tuttavia, al di là di queste accortezze minime, la sicurezza non è stata un obiettivo primario in fase di sviluppo. Come spesso accade nei progetti didattici o nei piccoli siti artigianali, diverse misure fondamentali sono assenti o implementate solo in parte. Le password, ad esempio, vengono memorizzate in chiaro nel database, senza alcun meccanismo di hashing. Ciò significa che un eventuale accesso al database comporterebbe la completa esposizione delle credenziali di tutti gli utenti.

Un altro aspetto critico riguarda l'output encoding: i contenuti forniti dagli utenti, come i tweet, vengono stampati a video senza alcun tipo di escaping o sanitizzazione. Questo apre la strada a possibili attacchi di tipo Cross-Site Scripting (XSS), in cui un utente malintenzionato potrebbe iniettare codice JavaScript all'interno della piattaforma, mettendo a rischio altri utenti.

Anche la gestione delle sessioni appare molto semplificata. Dopo l'autenticazione, l'ID di sessione non viene rigenerato, e non sono previsti timeout per l'inattività o protezioni contro il furto di cookie. Questo apre potenziali scenari di session hijacking, nei quali un attaccante può appropriarsi della sessione attiva di un utente e interagire con l'applicazione a suo nome. Inoltre, non sono stati implementati meccanismi di protezione contro attacchi CSRF (Cross-Site Request Forgery), rendendo potenzialmente vulnerabili tutte le operazioni effettuabili da un utente autenticato.

Infine, sebbene vengano utilizzati account separati per l'accesso al database, con privilegi minimi, il sistema non adotta controlli di autorizzazione approfonditi: alcune funzionalità si basano esclusivamente sulla fiducia nei parametri forniti dall'utente (es. ID utente o valori passati in POST), senza verificare in modo rigoroso che l'utente abbia effettivamente il diritto di accedere ad una determinata risorsa.

Nel complesso, l'applicazione si presenta come un buon esempio di web app "funzionante ma fragile": completa dal punto di vista delle funzionalità richieste dal progetto, ma ancora molto esposta sotto il profilo della sicurezza. Questa caratteristica la rende un terreno ideale per condurre un'analisi approfondita delle vulnerabilità, utilizzando strumenti automatici come OWASP ZAP. Le criticità rilevate in questa fase serviranno in seguito come punto di partenza per la successiva implementazione di contromisure appropriate.

3.2 Analisi Dinamica dell'Applicazione Twittle con OWASP ZAP

Dopo aver delineato il contesto tecnico e funzionale dell'applicazione Twittle, è stata avviata una fase di analisi dinamica mirata all'identificazione delle vulnerabilità più comuni tramite uno degli strumenti di sicurezza open-source più diffusi: OWASP ZAP (Zed Attack Proxy).

3.2.1 Configurazione del Proxy e Gestione del Traffico HTTPS

Per consentire a OWASP ZAP di analizzare in modo efficace le interazioni tra browser e server durante l'utilizzo dell'applicazione Twittle, è stato necessario configurare lo strumento come proxy locale, in grado di intercettare e gestire il traffico in entrata e in uscita. In particolare, si è resa necessaria l'abilitazione dell'intercettazione sia del traffico HTTP che di quello HTTPS, attraverso un'opportuna configurazione del sistema operativo e del browser utilizzato per l'analisi, ovvero Google Chrome.

OWASP ZAP agisce come un intermediario (proxy) tra client e server: tutte le richieste generate dal browser vengono instradate verso ZAP, che le riceve, le registra e, se necessario, le modifica prima di inoltrarle al server di destinazione. Per impostazione predefinita, il proxy locale di ZAP è in ascolto sull'indirizzo localhost (127.0.0.1) e sulla porta 8080. La configurazione è accessibile dal percorso interno Strumenti → Opzioni → Network → Local Proxies [9].

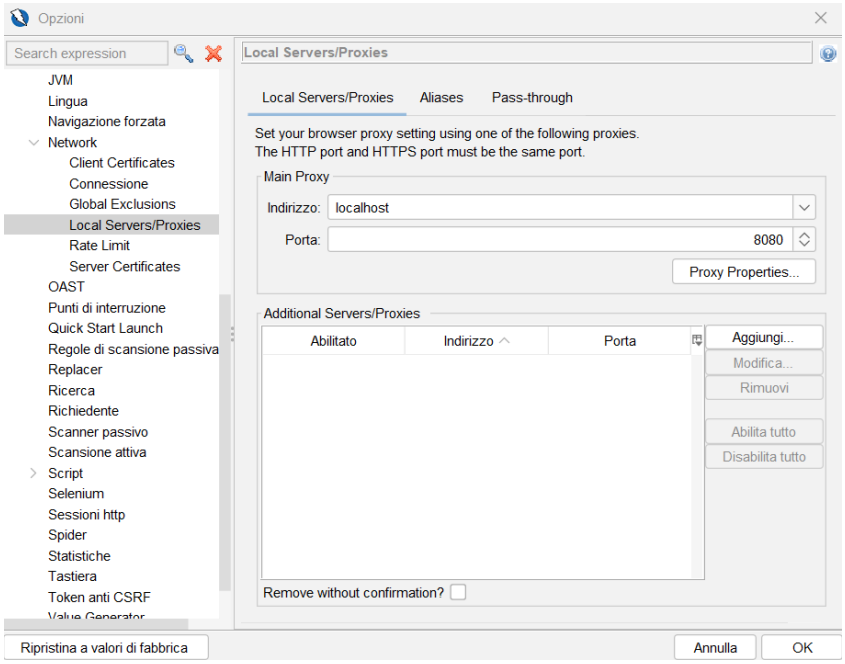


Immagine 3.1*: Local Proxies di ZAP*

Affinché il traffico web del browser venga effettivamente deviato attraverso il proxy locale, è stato necessario intervenire manualmente sulle impostazioni di rete del sistema operativo. Di seguito sono riportati i passaggi seguiti per la configurazione del proxy manuale in ambiente Windows 11:

1. All'interno di Google Chrome, si è aperto il menu principale (icona con tre punti verticali, in alto a destra) e selezionata la voce "Impostazioni".
2. Dal menu laterale, è stata selezionata la sezione "Sistema", quindi è stato cliccato su "Apri le impostazioni proxy del computer", che reindirizza alle impostazioni di rete del sistema operativo.
3. Si è aperta la finestra Impostazioni → Rete e Internet → Proxy.
4. All'interno della sezione "Configurazione manuale proxy", è stato cliccato su "Configura".
5. È stato attivato l'interruttore "Usa un server proxy", specificando:
    a. Indirizzo: 127.0.0.1
    b. Porta: 8080
6. Infine, la configurazione è stata salvata cliccando sul pulsante "Salva".

Una volta completata questa configurazione, tutto il traffico generato dalle applicazioni che utilizzano la rete, incluso quello del browser Chrome, viene automaticamente instradato attraverso il proxy locale OWASP ZAP. Questo consente allo strumento di monitorare e analizzare in tempo reale le richieste e risposte HTTP/HTTPS, prerequisito fondamentale per eseguire test dinamici di sicurezza sull'applicazione web analizzata.

3.2.2 Intercettazione del Traffico HTTPS e Gestione del Certificato CA

A differenza delle connessioni HTTP, il traffico HTTPS è cifrato mediante il protocollo TLS. Per permettere a ZAP di ispezionare anche questo tipo di traffico, è stato necessario importare nel browser il certificato di Autorità di Certificazione (CA) generato da ZAP stesso. Questo passaggio consente a ZAP di decifrare le comunicazioni cifrate tra browser e server, agendo come un intermediario "attendibile".

Il processo di importazione del certificato è avvenuto come segue:

1. All'interno di ZAP, dalla sezione "Strumenti → Opzioni → Network → Server Certificates"
2. Da questa schermata, è stato esportato in locale il certificato CA in formato .cer, cliccando sul pulsante "Salva".
3. In Chrome, selezionando Impostazioni → Privacy e sicurezza → Sicurezza → scorrere in basso e cliccare su Gestisci certificati → Gestisci certificati importati da Windows → Autorità di certificazione radice attendibili
4. Cliccare "importa" in basso a sinistra.
5. Si aprirà la procedura guidata di importazione certificati, dove verrà richiesto di selezionare il file .cer.
6. Confermare l'aggiunta del certificato. Il certificato apparirà nella lista e sarà considerato attendibile per Chrome.

3.2.3 Inizio dell'Analisi

La web app Twittle oggetto dell'analisi è accessibile all'indirizzo http://195.231.2.153.sslip.io/s286979/website/PWR_twittle/. L'ambiente di test è stato predisposto su una macchina locale con sistema operativo Windows 11, utilizzando il browser Chrome e l'ultima versione disponibile di OWASP ZAP al momento dell'analisi (v2.16.1).

Per quanto riguarda la metodologia di scansione, è stato adottato un approccio ibrido, che ha combinato:

- *Esplorazione manuale* ("*Manual Explore*"): questa fase è stata eseguita dopo la corretta configurazione del proxy e l'importazione del certificato CA nel browser, permettendo a OWASP ZAP di intercettare e registrare anche il traffico HTTPS. L'analista ha navigato manualmente il sito visitando tutte le pagine significative dell'applicazione, quali registrazione, login, scrivi, bacheca, logout e consultazione dei contenuti pubblici in "scopri". Durante questa fase, OWASP ZAP ha operato in modalità proxy, intercettando e registrando tutte le richieste HTTP/HTTPS generate dal browser. L'esplorazione manuale si è rivelata particolarmente utile per mappare correttamente endpoint protetti da autenticazione o attivati da specifiche interazioni utente, superando i limiti dello spider automatico di ZAP, che spesso non riesce a gestire pagine dinamiche, form autenticati o JavaScript complesso.

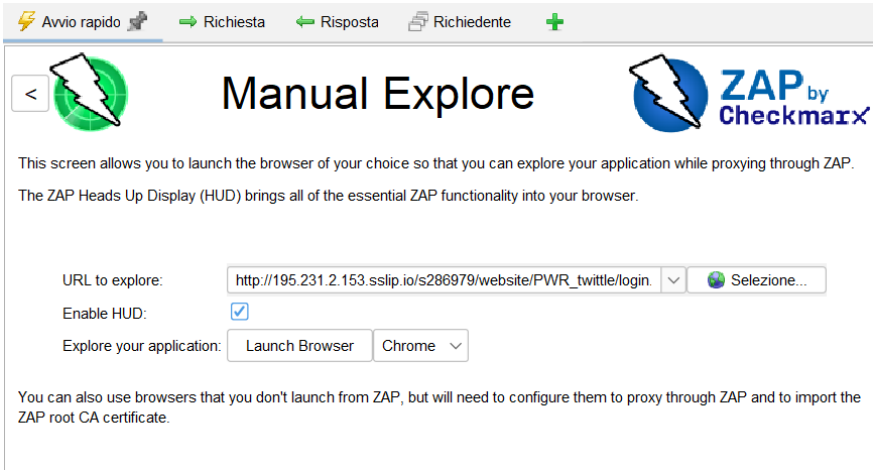


Immagine 3.2: *Manual Explore di OWASP ZAP*

- *Scansione attiva* ("*Active Scan*"): durante questa fase, è stato possibile definire uno scope specifico per la scansione, escludendo pagine o risorse non rilevanti per l'analisi. Nel caso specifico dell'applicazione Twittle, è stata selezionata l'intera cartella PWR_Twittle per garantire che tutte le funzionalità pertinenti venissero testate. Per avviare la scansione, è stato sufficiente cliccare con il tasto destro sul contesto desiderato, selezionare "Attacco" e poi "Scansione attiva".
  La scansione attiva ("*Active Scan*"), avviata al termine della fase di mappatura, ha comportato l'invio da parte di ZAP di una serie di richieste modificate verso le risorse individuate. Queste richieste includevano payload malevoli progettati per testare la presenza di vulnerabilità comuni, come SQL injection, Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF), directory traversal, e problematiche legate all'autenticazione o alla gestione della sessione.

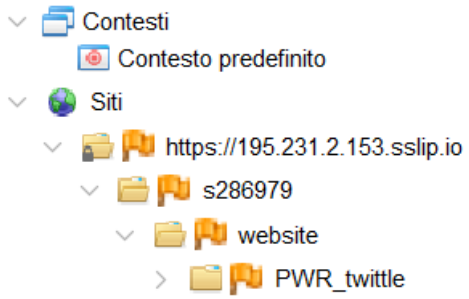


Immagine 3.3: *Contesto scelto*

L'integrazione tra esplorazione manuale e scansione attiva ha permesso di ottenere una fotografia realistica della superficie d'attacco esposta dall'applicazione, includendo anche quelle aree non raggiungibili tramite scansioni completamente automatiche.

3.2.4 Scelta del Criterio di Scansione

Per ottenere risultati significativi e coerenti con le caratteristiche dell'applicazione Twittle, è stato necessario configurare attentamente la *Scan Policy* più adatta applicabile alla fase di Scansione Attiva.

OWASP ZAP offre infatti diversi profili preconfigurati di scansione, ciascuno pensato per un contesto d'uso specifico. Queste configurazioni determinano il livello di profondità, aggressività, tempo richiesto e tipo di test eseguiti, permettendo così di adattare l'analisi al ciclo di vita del software o all'obiettivo specifico del test (sviluppo, test QA, CI/CD, penetration testing, ecc.) [10].

Di seguito viene presentata una panoramica dei principali profili disponibili:

Profilo di Scansione Attiva	Descrizione	Durata stimata	Copertura	Contesto consigliato
Default	Configurazione standard con un insieme bilanciato di test	Media (~20 min)	Media	Test preliminari o generici
API Scan	Ottimizzato per analisi di API REST/JSON	Variabile	Alta (solo API)	Applicazioni API-centriche
Developer CI/CD	Scansione rapida e leggera per pipeline di integrazione continua	Bassa	Media	Automazione in ambienti CI/CD
Developer Standard	Set di test più estesi, adatto a un uso quotidiano da parte dei team di sviluppo	Media	Alta	Ambiente di sviluppo
Developer Full	Tutti i plugin attivi, massima profondità e copertura	Alta (~60+ min)	Molto alta	Analisi completa pre-rilascio
Penetration Tester	Profilo molto aggressivo con payload avanzati	Molto alta	Massima	Simulazione di attacchi reali
QA Standard / Full	Verifiche sulla stabilità e coerenza dell'app in ambienti di quality assurance	Media / Alta	Alta	QA testing prima di produzione

Tabella 3.1: *Confronto tra i diversi profili di Scan Policy*

Nel contesto dell'analisi di Twittle è stato selezionato il profilo Developer Full, poiché rappresenta il compromesso ideale tra tempo di scansione e completezza dell'analisi, garantendo un controllo dettagliato su tutte le superfici esposte, inclusi i form autenticati e le funzionalità interattive.

Questo profilo attiva tutti i plugin disponibili per la scansione attiva, tra cui test per:

- Injection (SQL, Command, LDAP)
- Cross-Site Scripting (riflesso, persistente, DOM-based)
- Scoperta di file e directory nascoste
- Test su header HTTP insicuri o mancanti
- Vulnerabilità nella gestione delle sessioni e nei cookie
- Rischi legati a configurazioni server deboli o errori comuni

Anche se la durata della scansione è generalmente superiore rispetto ai profili base, l'utilizzo del profilo Developer Full ha permesso di individuare una serie di vulnerabilità complesse che un'analisi più rapida avrebbe potuto trascurare. Nel caso specifico di Twittle, tuttavia, questo impatto sul tempo è stato marginale: si tratta infatti di una web application di dimensioni contenute, con un numero limitato di pagine e funzionalità ben definite. Questo ha reso possibile eseguire una scansione completa e approfondita in tempi comunque accettabili (inferiori ai trenta minuti), beneficiando al massimo della copertura estesa offerta dal profilo scelto. La presenza di form autenticati, interazioni tra utenti e gestione delle sessioni ha comunque richiesto un'analisi accurata, per cui la scelta di un profilo completo si è rivelata proporzionata e vantaggiosa rispetto alla complessità dell'applicazione.

3.3 Vulnerabilità Rilevate

Al termine della scansione attiva descritta in precedenza, ZAP ha generato un report strutturato di alert classificati per livello di rischio in base al colore assegnato. Il contrassegno rosso indica problemi di gravità elevata; l'arancione segnala vulnerabilità di gravità media; il giallo rappresenta casi di bassa gravità; infine, il blu identifica le informazioni (non necessariamente problemi di sicurezza).

Di seguito vengono mostrati i risultati restituiti da ZAP:

Nome	Livello di Rischio	Number of Instances
Cross Site Scripting (Persistent)	Alto	2
Cross Site Scripting (Reflected)	Alto	1
Assenza di Token Anti-CSRF	Medio	7
Content Security Policy (CSP) Header Not Set	Medio	9
Directory Browsing	Medio	4
Missing Anti-clickjacking Header	Medio	9
Cookie No HttpOnly Flag	Basso	4
Cookie Without Secure Flag	Basso	4
Cookie without SameSite Attribute	Basso	4
Server Leaks Information via "X-Powered-By" HTTP Response Header Field(s)	Basso	11
Server Leaks Version Information via "Server" HTTP Response Header Field	Basso	12
Strict-Transport-Security Header Not Set	Basso	10
X-Content-Type-Options Header Missing	Basso	10
Authentication Request Identified	Informativo	3
Cookie Poisoning	Informativo	3
Information Disclosure - Suspicious Comments	Informativo	1
Modern Web Application	Informativo	11
Session Management Response Identified	Informativo	5
User Controllable HTML Element Attribute (Potential XSS)	Informativo	8

Immagine 3.4: *Elenco avvisi di sicurezza*

3.4 Vulnerabilità ad Alto Rischio (Rosse)

3.4.1 Cross Site Scripting (Persistent)

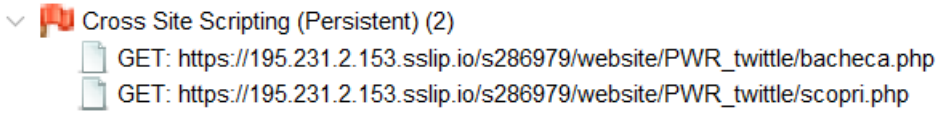


Immagine 3.4: Avviso di Cross Site Scripting Persistente

- Pagine coinvolte: `scopri.php`, `bacheca.php`
- Parametro vulnerabile: tweet, username
- Gravità: Alta. Esecuzione automatica di codice lato client per tutti gli utenti

Il Cross-Site Scripting (XSS) di tipo persistente rappresenta una delle vulnerabilità più gravi in ambito web, in quanto consente a un attaccante di iniettare script maligni nei contenuti di un'applicazione, i quali vengono successivamente salvati e visualizzati automaticamente nei browser degli altri utenti.

Nel caso specifico dell'applicazione Twittle, è stato rilevato che tramite la pagina scrivi.php è possibile immettere codice JavaScript all'interno del campo tweet, il quale viene successivamente riproposto nelle pagine pubbliche (scopri.php, bacheca.php) senza alcun meccanismo di sanificazione o escaping.

Un esempio di payload rilevato è stato:

```
</p><script>alert(1);</script><p>
```

Durante la scansione attiva, ZAP ha iniettato diversi payload tipici da XSS all'interno dei parametri rilevati. Dopo l'invio del contenuto tramite il form di scrittura, lo script è stato ritrovato nella risposta della pagina pubblica.

Possibili contromisure:

- Effettuare *escaping* dei caratteri speciali nel contenuto visualizzato
- Sanificare i dati in ingresso e in uscita, eliminando tag HTML non previsti
- Aggiungere l'header Content-Security-Policy per limitare le fonti autorizzate all'esecuzione di script
- Impostare i cookie di sessione con i flag HttpOnly, Secure e SameSite per evitare che vengano letti o inviati tramite script

3.4.2 Cross Site Scripting (Reflected)

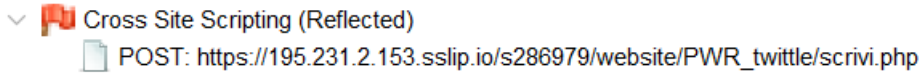


Immagine 3.5: Avviso di Cross Site Scripting Riflesso

- Pagina coinvolta: `scrivi.php`
- Metodo: POST
- Gravità: Alta. Potenziale sfruttamento tramite link appositamente costruiti

A differenza dell'XSS persistente, la variante riflessa si manifesta quando un input malevolo non viene salvato sul server, ma viene immediatamente riflesso nella risposta della pagina. Questo consente a un attaccante di costruire URL malevoli contenenti script, che, una volta cliccati da una vittima, causano l'esecuzione di codice arbitrario nel browser. Nel caso di Twittle, è stato possibile inserire un payload come:

```
</p><scrIpt>alert(1);</scRipt><p>
```

Lo script è stato poi visualizzato direttamente nel corpo della risposta HTTP, confermando la vulnerabilità.

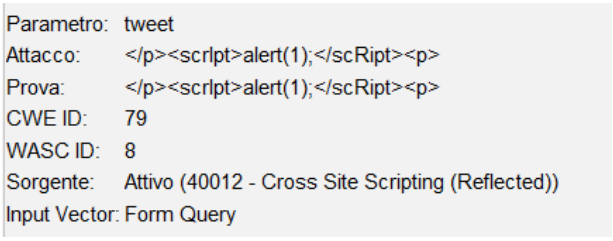


Immagine 3.6: *Test vulnerabilità XSS*

Possibili contromisure:

- Applicare output *encoding* su tutti i dati riflessi dinamicamente nelle risposte
- Limitare i contenuti dinamici generati da input utente
- Aggiungere un'adeguata Content Security Policy
- Validare e filtrare gli input anche lato server, per evitare l'iniezione di codice HTML o JS

3.5 Vulnerabilità a Medio Rischio (Arancioni)

3.5.1 Assenza di Token Anti-CSRF



Immagine 3.7: Avviso di assenza di Token Anti-CSRF

- Pagine coinvolte: `login.php`, `registrazione.php`, `scrivi.php`, `bacheca.php`
- Gravità: Media. Rischio di azioni involontarie eseguite da utenti autenticati

La mancanza di protezione contro il Cross-Site Request Forgery (CSRF) è una criticità ricorrente nell'applicazione. In questo tipo di attacco, un utente autenticato può essere ingannato ad eseguire un'azione non intenzionale semplicemente visitando una pagina o cliccando su un link creato da un attaccante. Il problema nasce dal fatto che le richieste POST non includono alcun token univoco legato alla sessione o all'utente. L'applicazione accetta e processa qualsiasi richiesta proveniente dal browser della vittima, senza verificarne la legittimità.

Possibili contromisure:

- Integrare un token univoco (anti-CSRF) all'interno di ogni form HTML
- Verificare lato server la validità del token in ogni richiesta che modifica lo stato dell'applicazione

- Impostare i cookie di sessione con attributo `SameSite=Strict` per limitare l'invio automatico da siti esterni

3.5.2 Content Security Policy (CSP)



Immagine 3.8: Avviso di assenza di Content Security Policy Header

- Impatto su tutta l'applicazione
- Gravità: Media. Aumenta l'efficacia degli attacchi XSS e simili

La mancanza di Content Security Policy rappresenta una vulnerabilità di configurazione che amplifica significativamente l'impatto di altre vulnerabilità, in particolare quelle relative al Cross Site Scripting. Il CSP agisce come una linea di difesa avanzata, definendo le sorgenti autorizzate per l'esecuzione di script, il caricamento di risorse e altre operazioni potenzialmente pericolose.

Questa problematica si manifesta in modo pervasivo attraverso tutta l'applicazione, difatti nessuna pagina implementa questa importante misura di sicurezza. L'assenza di CSP significa che, in caso di successo di un attacco XSS, l'attaccante ha piena libertà di eseguire script arbitrari, caricare risorse esterne, e potenzialmente rubare dati sensibili.

Possibili contromisure:

- Aggiungere nella risposta HTTP un header CSP restrittivo
- Evitare l'uso di script inline

3.5.3 Directory Browsing

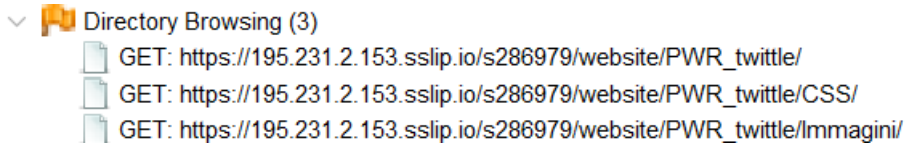


Immagine 3.9: Avviso possibilità di Directory Browsing

- Directory interessate: alcune directory interne raggiungibili via browser
- Gravità: Media. Esposizione della struttura interna dell'applicazione

Il directory browsing si verifica quando il server web consente la visualizzazione del contenuto delle directory in cui non è presente un file predefinito (come `index.html` o `index.php`). In tali casi, il server risponde mostrando automaticamente l'elenco dei file e delle sottocartelle presenti nella directory richiesta.

Nel contesto dell'applicazione Twittle, ZAP ha rilevato che alcune directory risultano accessibili e indicizzabili da utenti non autenticati, consentendo la navigazione libera del file system dell'applicazione. Questo comportamento può esporre file temporanei, risorse interne, script non destinati al pubblico, o persino dati sensibili. Anche se a prima vista può sembrare una vulnerabilità di basso impatto, in realtà costituisce una porta di accesso informativa importante per un attaccante, che può usarla per:

- Mappare la struttura interna del progetto
- Individuare nomi di file potenzialmente vulnerabili
- Scaricare file che potrebbero contenere credenziali, configurazioni, backup o codice sorgente

Possibili contromisure:

- Disabilitare l'indicizzazione delle directory dal file di configurazione del server web. Per Apache, aggiungere la direttiva: `Options -Indexes.`

3.5.4 Assenza di Anti-clickjacking Header

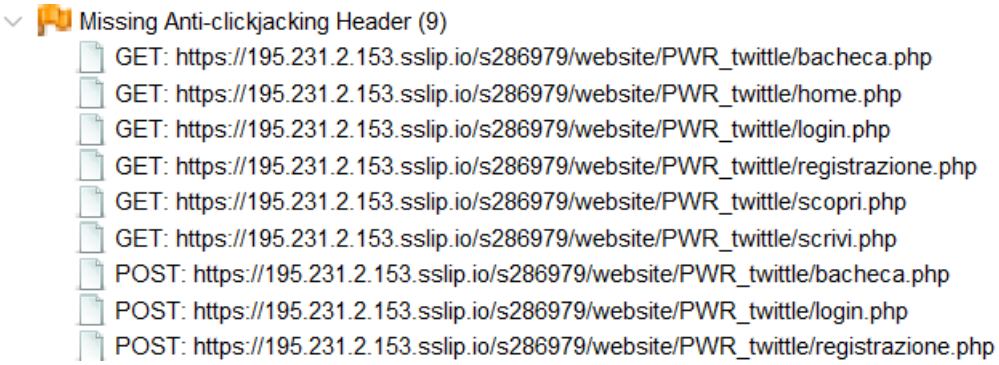


Immagine 3.10: Avviso di assenza di Anti-clickjacking Header

- Pagine interessate: tutte le principali (`login.php`, `scrivi.php, bacheca.php`, ecc.)
- Gravità: Media. Esposizione ad attacchi di tipo clickjacking

Risulta inoltre assente l'header anti-clickjacking (`X-Frame-Options` o `Content-Security-Policy` con `frame-ancestors`) che rischia di esporre l'applicazione ad attacchi di clickjacking, una tecnica che permette agli attaccanti di indurre gli utenti a cliccare su elementi nascosti o camuffati.

Questa vulnerabilità si manifesta su tutte le pagine principali dell'applicazione. Un attaccante potrebbe incorporare pagine di Twittle all'interno di iframe invisibili o trasparenti, sovrapponendoli a contenuti apparentemente innocui. Quando l'utente interagisce con quello che crede essere contenuto legittimo, in realtà sta eseguendo azioni sull'applicazione Twittle.

ZAP ha analizzato le intestazioni di risposta HTTP e ha segnalato che nessuna delle pagine testate includeva l'header X-Frame-Options. Inoltre, anche il campo Content-Security-Policy (che può contenere direttive equivalenti tramite frame-ancestors) risulta completamente assente, aggravando ulteriormente il problema.

Possibili contromisure:

- Impedire completamente il caricamento in iframe: `X-Frame-Options: DENY.`
- Consentire solo il framing da parte dello stesso dominio: `X-Frame-Options: SAMEORIGIN.`

- Oppure inserire `Content-Security-Policy: frame-ancestors 'self'.`

In generale, per applicazioni che non richiedono esplicitamente l'utilizzo di iframe, è buona prassi bloccarlo del tutto, prevenendo così ogni forma di attacco clickjacking in maniera semplice ed efficace.

3.6 Vulnerabilità a Basso Rischio (Gialle)

3.6.1 Problematiche di configurazione dei Cookie



Immagine 3.11: Avviso di assenza di flag di sicurezza

- Cookie interessati: `PHPSESSID`, `last_username`
- Gravità: Bassa, ma potenzialmente sfruttabile in combinazione con altre vulnerabilità

Durante l'analisi sono stati identificati alcuni cookie utilizzati dall'applicazione che non includono i flag di sicurezza raccomandati. In particolare, il cookie PHPSESSID, utilizzato per gestire la sessione dell'utente e il cookie `last_username`, che memorizza l'ultimo nome utente inserito nei moduli di login/registrazione. Entrambi risultano privi dei seguenti attributi:

- HttpOnly: impedisce l'accesso al cookie via JavaScript, utile contro XSS
- Secure: garantisce che il cookie sia trasmesso solo su connessioni HTTPS
- SameSite: limita l'invio del cookie in contesti cross-site, utile contro CSRF

Questa cattiva configurazione, sebbene da sola non rappresenti una falla critica, aumenta significativamente il rischio se combinata con altre vulnerabilità più gravi, come XSS persistente o assenza di token CSRF.

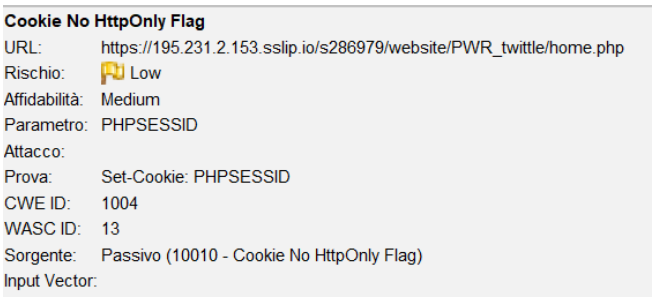


Immagine 3.12: *Avviso di assenza di flag HttpOnly in home.php*

Possibili contromisure:

- Configurare i cookie con flag di sicurezza adeguati.
- Impiegare sessioni di breve durata e logout automatici per ridurre la finestra di esposizione

4.6.2 Esposizione di Informazioni Tecniche

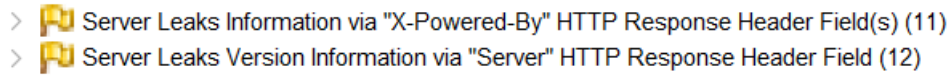


Immagine 3.13: Esposizione di informazioni tecniche

- Header esposti: `X-Powered-By, Server`
- Gravità: Bassa. Rischio di information disclosure utile per attacchi mirati

L'esposizione di informazioni tecniche attraverso header HTTP rappresenta un problema di *information disclosure* che facilita significativamente la raccolta di informazioni da parte di un potenziale attaccante. L'header `X-Powered-By` rivela la tecnologia utilizzata dal server, in questo caso PHP/7.4.3, fornendo agli attaccanti informazioni precise sullo stack applicativo. Questa informazione permette a un attaccante di concentrare le proprie attività di ricerca su vulnerabilità note relative alla versione individuata, consultando fonti pubbliche come il database CVE (*Common Vulnerabilities and Exposures*). Nel caso di PHP, versioni non aggiornate possono presentare vulnerabilità già documentate ed eventualmente corredate di exploit, aumentando così il rischio di compromissione dell'applicazione.

Similmente, la divulgazione di informazioni sulla versione del server web, ovvero *nginx/1.15.12*, consente a un potenziale attaccante di individuare vulnerabilità note collegate a versioni specifiche di tali software. Questo tipo di informazione è particolarmente rilevante poiché i server web costituiscono la prima linea di esposizione delle applicazioni verso l'esterno.

4.6.3 Assenza di Strict-Transport-Security Header

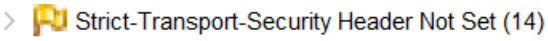


Immagine 3.14: Header HSTS mancante

- Gravità: Bassa, ma cruciale per garantire connessioni sicure nel tempo

La mancata configurazione dell'header HTTP Strict Transport Security (HSTS) evidenzia una debolezza di configurazione comune a tutta l'applicazione. Sebbene il sito disponga di un certificato TLS valido e supporti connessioni cifrate HTTPS, l'assenza di HSTS impedisce di forzare i browser a utilizzare esclusivamente connessioni sicure per tutte le richieste future.

Senza HSTS, l'applicazione rimane esposta a potenziali attacchi di downgrade, in cui un attaccante presente sulla rete (ad esempio su una rete Wi-Fi pubblica) potrebbe indurre il browser a stabilire la connessione tramite HTTP non cifrato, intercettando così dati sensibili. Allo stesso modo, si amplifica il rischio di SSL stripping, tecnica che rimuove la cifratura HTTPS dalle comunicazioni tra utente e server.

Questo tipo di vulnerabilità è particolarmente insidioso perché spesso non viene percepito dall'utente finale, che non nota l'assenza del canale cifrato. In un contesto come Twittle, dove vengono gestiti contenuti personali, credenziali e cookie di sessione, la mancanza di HSTS rappresenta un fattore di rischio concreto per la confidenzialità e l'integrità delle informazioni scambiate.

4.6.4 Assenza di X-Content-Type-Options Header

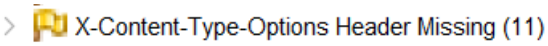


Immagine 3.15: Header X-Content-Type-Options mancante

- Gravità: Bassa, ma può contribuire ad esecuzioni XSS involontarie

L'assenza dell'header `X-Content-Type-Options` rappresenta una vulnerabilità di configurazione che, seppur spesso trascurata, può avere ripercussioni concrete sulla sicurezza dell'applicazione. Con 11 occorrenze rilevate, questa problematica risulta diffusa su diverse pagine del sito.

Impostare l'header con valore `nosniff` istruisce i browser a non tentare di "indovinare" il tipo di contenuto di una risorsa, affidandosi esclusivamente all'header Content-Type dichiarato dal server. In assenza di questa direttiva, i browser possono interpretare contenuti testuali in modi non intenzionali, introducendo rischi di esecuzione di codice indesiderato.

Nel caso di *Twittle*, sebbene l'applicazione non consenta l'upload di file binari o multimediali, gli utenti possono comunque immettere contenuti testuali potenzialmente manipolati, come tweet o messaggi. Senza `X-Content-Type-Options: nosniff`, un contenuto restituito dal server potrebbe essere interpretato come script eseguibile, favorendo l'exploit di vulnerabilità di tipo XSS.

In un contesto dove sono già state riscontrate debolezze XSS, la mancata implementazione di questo header contribuisce ad amplificare la superficie di attacco, rendendo più semplice aggirare altre misure di filtraggio o sanificazione degli input.

3.7 Informazioni (Blu)



Immagine 3.16: avvisi informativi

Authentication Request Identified: questa segnalazione non rappresenta una vulnerabilità diretta, ma evidenzia la presenza di parametri sensibili, come i campi password nei moduli di autenticazione. Indica che tali punti meritano un controllo accurato per garantire che i dati immessi siano protetti da meccanismi adeguati di cifratura in transito (TLS) e da eventuali attacchi di tipo brute force o credential stuffing. Inoltre, è buona pratica limitare l'utilizzo di nomi di parametri e ID troppo espliciti, prediligendo denominazioni meno riconoscibili, così da rendere più complesso per un potenziale attaccante individuare automaticamente i punti di ingresso più sensibili.

Cookie Poisoning: il rischio di *cookie poisoning* deriva dalla possibilità che un attaccante modifichi i valori di cookie memorizzati lato client per manipolare lo stato dell'applicazione. È quindi fondamentale validare sempre lato server i cookie ricevuti, evitando di fare affidamento su valori che potrebbero essere alterati.

Information Disclosure – Suspicious Comments: la presenza di commenti o frammenti di markup potenzialmente informativi, come tag `<span class="username">`, può fornire indizi utili a un attaccante per comprendere la struttura interna dell'interfaccia utente o la logica di gestione degli account. Anche se questi dettagli non costituiscono una falla immediata, è buona pratica limitare l'esposizione di informazioni superflue nel codice HTML pubblico.

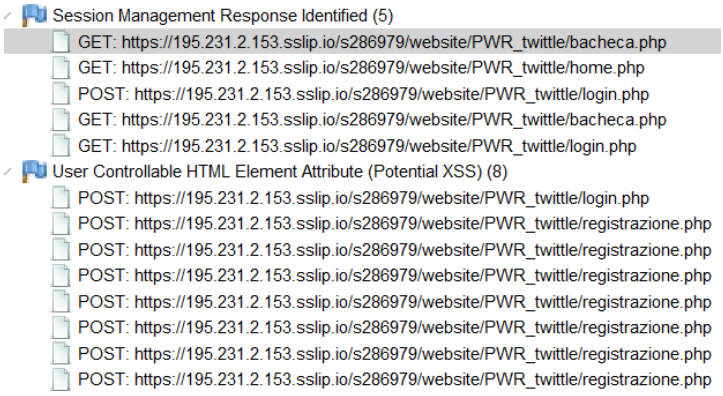


Immagine 3.17: avvisi informativi

Session Management Response Identified: come per l'autenticazione, questa segnalazione evidenzia la presenza e l'utilizzo del cookie di sessione PHPSESSID all'interno dell'applicazione. Sebbene non rappresenti una vulnerabilità diretta, l'uso di nomi standard e facilmente riconoscibili per i cookie di sessione può agevolare un potenziale attaccante nelle attività di fingerprinting e nello sviluppo di attacchi mirati, come session hijacking. È buona pratica adottare nomi di cookie di sessione meno prevedibili e generici, insieme a una corretta configurazione dei flag di sicurezza (`HttpOnly`, `Secure`, `SameSite`), per ridurre la superficie di attacco e proteggere meglio le sessioni utente.

User Controllable HTML ELement Attribute (Potential XSS): ZAP ha rilevato che in alcune pagine dell'applicazione Twittle l'input dell'utente viene inserito direttamente all'interno di attributi HTML (come `href`, `src`, o `title`) senza un'adeguata codifica. Questo comportamento può creare le condizioni per un attacco XSS, specialmente se l'attributo è interattivo o legato a eventi JavaScript. Anche se durante i test non è stato eseguito codice dannoso, l'inserimento non filtrato in un attributo può permettere, ad esempio, a un attaccante di interrompere l'attributo e iniettare un `onmouseover="..."` o uno script `javascript:`.

3.8 Analisi dell'Impatto Complessivo

L'analisi complessiva delle vulnerabilità rivela un quadro preoccupante dal punto di vista della sicurezza. La presenza simultanea di vulnerabilità XSS persistenti e riflesse, combinata con l'assenza di misure di protezione fondamentali come CSP, CSRF token, e configurazioni sicure dei cookie, crea un ambiente particolarmente favorevole per gli attaccanti.

La distribuzione delle vulnerabilità mostra come problematiche di configurazione e mancanza di best practice di sicurezza si manifestino in modo sistematico attraverso l'applicazione. Il numero elevato di occorrenze per vulnerabilità come l'assenza di header di sicurezza (14 occorrenze per HSTS, 11 per X-Content-Type-Options, 10 per CSP) indica che questi problemi non sono isolati ma rappresentano lacune strutturali nell'approccio alla sicurezza.

Particolarmente allarmante è la combinazione tra vulnerabilità XSS e l'assenza di protezioni dei cookie: un attaccante che riesca a sfruttare le vulnerabilità di scripting avrebbe accesso diretto ai cookie di sessione, potendo così compromettere completamente l'account delle vittime.

La classificazione finale del rischio per l'applicazione Twittle deve essere considerata alta, principalmente a causa della presenza di vulnerabilità XSS persistenti che, in combinazione con le altre problematiche identificate, potrebbero essere sfruttate per compromettere sistematicamente la sicurezza degli utenti dell'applicazione.

Capitolo 4

Implementazione delle Tecniche di Mitigazione

Una volta completata l'analisi iniziale dell'applicazione *Twittle*, è stato possibile definire un insieme chiaro di vulnerabilità da correggere. I problemi emersi, rilevati sia tramite code review strutturata secondo le linee guida OWASP sia attraverso l'analisi dinamica con OWASP ZAP, hanno messo in evidenza diverse carenze a livello di sicurezza applicativa, in parte dovute alla natura didattica del progetto e in parte a scelte di sviluppo comuni nei contesti non produttivi.

Questo capitolo si concentra sull'intervento correttivo, illustrando le modifiche apportate per ridurre o eliminare i rischi individuati. L'approccio seguito è stato sistematico e ha preso come riferimento le principali categorie di vulnerabilità trattate nel Capitolo 2, che hanno offerto una base metodologica solida per guidare l'intera attività di mitigazione.

Ogni sezione del capitolo documenta una problematica specifica, descrivendo in modo chiaro il contesto, le motivazioni alla base della soluzione scelta e le modifiche implementate. L'obiettivo non è soltanto quello di incrementare il livello di sicurezza dell'applicazione, ma anche di favorire una maggiore consapevolezza sulle implicazioni pratiche delle vulnerabilità, evidenziando quanto sia importante integrare la sicurezza fin dalle prime fasi dello sviluppo.

4.1 Struttura Logica dell'Applicazione e Controlli di Sicurezza

Ai fini di garantire una maggiore sicurezza lato server e una chiara separazione tra il codice pubblico e quello privato, si è adottata un'architettura progettuale suddivisa logicamente in due aree distinte:

- `public/`: rappresenta l'area pubblica dell'applicazione web, direttamente accessibile dal browser. Essa ospita le pagine web visibili dall'utente finale, i fogli di stile, gli script client-side e le risorse multimediali necessarie al corretto rendering dell'interfaccia.
- `app/`: costituisce invece l'area riservata del progetto, non accessibile direttamente tramite browser. Al suo interno risiedono i database (organizzati in `db_normale/` e `db_privilegiato/` a seconda del livello di accesso richiesto), i file di log e tutte le componenti sensibili dell'applicazione, tra cui i file di gestione delle sessioni e di configurazione.

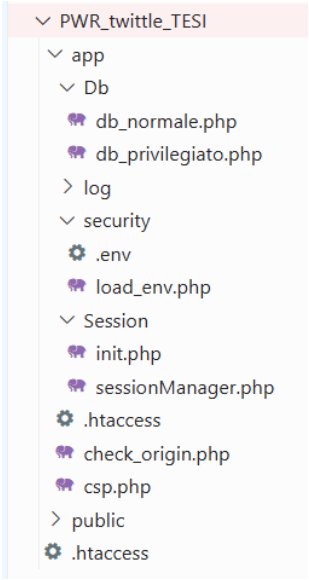


Immagine 4.1: *Struttura interna del progetto*

4.1.1 Sicurezza Mediante Configurazione .htaccess

Per rafforzare ulteriormente la protezione delle aree critiche dell'applicativo, sono stati adottati specifici accorgimenti attraverso l'utilizzo dei file `.htaccess`, tipici degli ambienti Apache [11].

In particolare, all'interno della directory `app/` è stato predisposto un file `.htaccess` il cui scopo principale è quello di bloccare completamente l'accesso diretto tramite browser. Questa misura si rende necessaria per tutelare il codice sorgente, le credenziali di accesso e i file di log da eventuali richieste non autorizzate. L'accesso a questa cartella, quindi, avviene esclusivamente tramite script lato server, evitando esposizioni involontarie dei dati sensibili.

```
<IfModule mod_authz_core.c>
  Require all denied
</IfModule>

<IfModule !mod_authz_core.c>
  Deny from all
</IfModule>
```

All'interno della directory `public/`, invece, il file `.htaccess` svolge diverse funzioni volte a incrementare la sicurezza dell'area visibile al pubblico:

- Disabilitazione del directory listing: per rafforzare la sicurezza dell'applicazione, è stata disabilitata la funzionalità di *directory listing*, ovvero la possibilità per un utente di visualizzare l'elenco dei file presenti in una directory del server nel caso in cui manchi un file di indice (come `index.php`). Questa pratica previene l'esposizione accidentale di file sensibili o non destinati alla consultazione diretta, come script di inclusione, file di configurazione o backup del codice sorgente. La loro esposizione potrebbe infatti fornire ad un attaccante informazioni utili per compromettere il sistema.
- Protezione dei file nascosti e di configurazione: per evitare che vengano letti file potenzialmente sensibili, come gli stessi `.htaccess`, è stata utilizzata una direttiva che blocca l'accesso a qualsiasi file il cui nome inizi con un punto (.).
- Forzatura del protocollo HTTPS: è stato configurato un reindirizzamento automatico per tutte le richieste ricevute tramite HTTP verso la versione HTTPS del sito. Questa misura assicura che

l'intera comunicazione tra client e server avvenga su un canale cifrato, riducendo il rischio di intercettazione dei dati e prevenendo attacchi di tipo *man-in-the-middle*. L'adozione obbligatoria del protocollo sicuro rappresenta uno standard imprescindibile per la protezione della riservatezza e dell'integrità delle informazioni trasmesse.

```
# Disabilitazione del directory listing
Options -Indexes

# blocco dell'accesso ai file di configurazione
<FilesMatch "^\.">
    Require all denied
</FilesMatch>

# Viene verificato se HTTPS è disattivato o se l'intestazione X-Forwarded-
Proto è diversa da https
RewriteCond %{HTTPS} !=on
RewriteCond %{HTTP:X-Forwarded-Proto} !https

# Reindirizzamento a HTTPS
RewriteRule ^ https://%{HTTP_HOST}%{REQUEST_URI} [L,R=301]
```

- Blocco dell'accesso diretto a componenti inclusi: è stato impedito l'accesso alla cartella `Componenti/`, che contiene frammenti di codice come `menu.php`, `header.php` e `footer.php`, progettati per essere inclusi in altre pagine. Questi file non rappresentano contenuti completi dal punto di vista HTML e, se richiamati direttamente, potrebbero generare comportamenti imprevisti. Il blocco è quindi funzionale a garantire che tali componenti siano utilizzati esclusivamente nel contesto previsto.

```
#Componenti/.htaccess
<IfModule mod_authz_core.c>
  Require all denied
</IfModule>

<IfModule !mod_authz_core.c>
  Deny from all
</IfModule>
```

L'inserimento degli header di sicurezza è stato effettuato direttamente tramite la funzione `header()` all'interno del codice PHP. Tale scelta è stata dettata da alcune limitazioni riscontrate nell'uso del file `.htaccess`, verosimilmente dovute alla mancata attivazione del modulo `mod_headers` di Apache o alla presenza di un reverse proxy, come Nginx, che potrebbe ignorare o sovrascrivere gli header impostati a livello di configurazione del web server. Definire questi header lato server consente di stabilire precise policy di sicurezza, limitare il caricamento di risorse non autorizzate, prevenire attacchi di tipo clickjacking e contrastare l'esecuzione di codice non valido.



Immagine 4.2: *Pagina Forbidden*

4.2 Mitigazione delle Vulnerabilità di Tipo Injection

4.2.1 SQL Injection

Uno degli aspetti fondamentali nella progettazione sicura di applicazioni web riguarda la protezione contro le vulnerabilità di tipo SQL Injection, le quali possono compromettere l'integrità, la riservatezza e la disponibilità dei dati. Tali attacchi si basano sull'inserimento di comandi SQL arbitrari da parte dell'utente all'interno delle query, con l'obiettivo di manipolare il comportamento del database sottostante.

Nel progetto in esame, l'utilizzo delle query parametrizzate (*prepared statements*) è stato adottato sin dalle fasi iniziali dello sviluppo, rappresentando fin da subito una buona pratica di sicurezza. Tali tecniche sono state successivamente rafforzate e uniformate in tutti i file che gestiscono operazioni con il database, come parte di un intervento sistematico di mitigazione delle vulnerabilità.

Le query parametrizzate permettono di separare in modo netto la logica delle interrogazioni dai dati forniti dall'utente, impedendo così che input malevoli possano essere interpretati come parte del codice SQL. Questo approccio è stato applicato nei seguenti script:

- `login.php`
- `registrazione.php`
- `bacheca.php`
- `scrivi.php`
- `ultimo_twit.php`

```php
//--- login.php ---
// inclusione del file di connessione al database
include __DIR__ . '/../app/Db/db_normale.php';

 //Preparazione della query per selezionare un utente in base allo username
$stmt = mysqli_prepare($conn, "SELECT * FROM utenti WHERE username = ?");

//In caso di errore nella preparazione della query, viene registrato un errore e
l'utente viene reindirizzato
if(!$stmt){
   logErrore("Errore prepare login.php: " . mysqli_error($conn));
   $_SESSION['error_message'] = "Errore del server. Riprova più tardi.";
   header("Location: login.php");
   exit();
}

//Associazione del parametro alla query (tipo stringa)
mysqli_stmt_bind_param($stmt, "s", $username);

//Esecuzione della query con gestione dell'errore in caso di fallimento
```

```php
if(!mysqli_stmt_execute($stmt)){
    logErrore("Errore execute login.php: " . mysqli_stmt_error($stmt));
    $_SESSION['error_message'] = "Errore del server. Riprova più tardi.";
    header("Location: login.php");
    exit();
}
//Recupero del risultato della query eseguita
$result = mysqli_stmt_get_result($stmt);
```

Per il file `scopri.php`, invece, non è stato necessario adottare i *prepared statements*, in quanto la query contenuta non utilizza dati provenienti dall'utente né input esterno, e viene eseguita una sola volta. In tale contesto, l'uso di query parametrizzate non avrebbe apportato benefici significativi né in termini di sicurezza né di performance.

A complemento di queste misure lato database, è stata implementata una validazione dei dati in ingresso sia lato client (JavaScript) sia lato server (PHP), per assicurarsi che i dati ricevuti rispettino i formati attesi e per prevenire comportamenti imprevisti a livello applicativo.

4.2.2 Logging degli Errori

La gestione centralizzata e strutturata degli errori rappresenta un pilastro fondamentale per la sicurezza e la manutenzione dell'applicazione. È stato quindi introdotto un sistema di logging personalizzato, finalizzato a tracciare eventi anomali e facilitare l'analisi successiva di eventuali comportamenti sospetti o malfunzionamenti.

Per ogni errore rilevato, il sistema registra:

- La data e ora dell'evento (timestamp)
- L'indirizzo IP del client
- Il User-Agent (informazioni sul browser e dispositivo)
- L'URL richiesto
- Il messaggio di errore generato

Le informazioni vengono scritte in un file esterno (`errori.log`) utilizzando la funzione `error_log()` con il flag 3, che consente il logging verso un file specifico. Questo meccanismo è attivo per monitorare errori di varia natura, tra cui connessione al database e problemi nell'elaborazione delle query nei seguenti file: `login.php`, `registrazione.php`, `ultimo_twit.php`, `scrivi.php`, `scopri.php` e `bacheca.php`.

Inoltre, dove appropriato, è stato adottato il costrutto try-catch, che consente una gestione più ordinata e prevedibile delle eccezioni, migliorando la robustezza e la leggibilità del codice.

```php
// funtion_log.php
function logErrore($msg) {
    $timestamp = date("Y-m-d H:i:s");
    $ip = $_SERVER['REMOTE_ADDR'] ?? 'IP_UNKNOWN';
    $userAgent = $_SERVER['HTTP_USER_AGENT'] ?? 'UA_UNKNOWN';
    $url = $_SERVER['REQUEST_URI'] ?? 'URL_UNKNOWN';
    $percorso = __DIR__ . "/errori.log";
    error_log("[$timestamp] IP=$ip | URL=$url | UA=$userAgent | MSG=$msg\n", 3,
$percorso);
}
```

```
[2025-07-13 10:33:00] IP=172.17.0.1 | URL=/s286979/website/PWR_twittle_TESI/public/registrazione.php
| UA=Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36
| MSG=Errore execute registrazione.php: Data too long for column 'pwd' at row 1
```

Immagine 4.3: *Generazione di log*

4.2.3 Content Security Policy (CSP)

Per rafforzare la sicurezza lato client e prevenire attacchi di tipo Cross-Site Scripting (XSS) e altre forme di code injection, è stata implementata una *Content Security Policy* (CSP) tramite appositi header HTTP. La policy viene definita dinamicamente nel codice PHP attraverso la funzione `header()` e prevede l'inserimento di un *nonce*, un valore casuale, valido per una sola richiesta, che consente l'esecuzione controllata di script inline, senza compromettere l'integrità della pagina [12].

Per centralizzare e semplificare la gestione della CSP, è stato creato un file dedicato, `csp.php`, situato all'interno dell'area riservata (`app/`). Questo file definisce e invia la policy, compreso il *nonce* generato, e viene poi incluso in tutte le pagine pubbliche dell'applicazione che necessitano della protezione CSP. Tale approccio migliora la manutenibilità del codice e garantisce coerenza nell'applicazione delle regole di sicurezza.

```php
// Generazione di un nonce sicuro per la sessione corrente
$nonce = base64_encode(random_bytes(16));

// Definizione dell'header CSP
header("Content-Security-Policy: default-src 'self'; script-src 'self' 'nonce-
$nonce'; style-src 'self'; img-src 'self'; object-src 'none'; frame-ancestors
'none'; base-uri 'none'; form-action 'self';");
```

Questa configurazione specifica:

- `default-src 'self'`: imposta come sorgente predefinita esclusivamente il dominio corrente, impedendo il caricamento di risorse da origini esterne non esplicitamente autorizzate. Si tratta di una misura fondamentale per isolare l'applicazione da contenuti potenzialmente dannosi ospitati su domini di terze parti.

- `script-src 'self' 'nonce-$nonce'`: consente l'esecuzione di script soltanto se provenienti dallo stesso dominio o se dotati di un nonce valido. Questo meccanismo rende sicura l'eventuale presenza di script inline, ostacolando in modo efficace gli attacchi XSS, sia di tipo persistente che riflesso, che fanno leva sull'iniezione di codice JavaScript non autorizzato.

- `style-src 'self'`, `img-src 'self'`: limitano il caricamento rispettivamente di fogli di stile e immagini alla sola origine locale. In tal modo, si evita che contenuti visivi o stilistici vengano recuperati da fonti non controllate, riducendo il rischio di tracciamento o manipolazione dell'interfaccia.

- `object-src 'none'`: disabilita completamente l'utilizzo di oggetti plugin-based (come `<object>`, `<embed>`, `<applet>`), tecnologie considerate deprecate e spesso associate a vettori di attacco.

- `frame-ancestors 'none'`: impedisce il framing della pagina da parte di domini terzi, mitigando attacchi di tipo clickjacking.

- `base-uri 'none'`, `form-action 'self'`: vincolano l'elemento `<base>` e l'invio dei form al solo dominio originario, impedendo eventuali redirezioni non autorizzate o la modifica implicita dei percorsi relativi.

4.3 Session Management

La gestione sicura delle sessioni rappresenta un aspetto essenziale per garantire la protezione dei dati e dei privilegi utente all'interno di un'applicazione web [13]. Nella configurazione originaria del sistema, non erano previste fondamentali misure di sicurezza legate alla gestione delle sessioni. Proprio per questo motivo, durante l'attività di revisione e hardening dell'applicativo, sono stati introdotti interventi mirati, implementati principalmente in due file distinti: `app/Session/init.php` e `app/Session/sessionManager.php`.

Queste modifiche hanno permesso di mitigare vulnerabilità ben note, come session fixation, session hijacking e attacchi CSRF, garantendo una gestione più robusta e conforme alle best practice.

4.3.1 Sicurezza e Configurazione Iniziale della Sessione

In quest'ottica, è stato predisposto un modulo dedicato all'inizializzazione delle sessioni, contenuto nel file `app/Session/init.php`. Qui vengono definiti tutti quei parametri di sicurezza che stabiliscono regole chiare su come devono essere gestiti gli identificativi di sessione e i relativi cookie, riducendo così i margini di manomissione o intercettazione.

Elemento chiave di questa configurazione è l'impiego della funzione `ini_set()`, utilizzata per modificare dinamicamente alcune direttive di PHP direttamente all'avvio dell'applicazione. In particolare, sono stati definiti i seguenti parametri:

- `session.use_only_cookies` fa sì che l'ID di sessione venga trasmesso solo tramite cookie, evitando che possa finire nell'URL o in altri canali facilmente intercettabili, come i log del server o la cronologia del browser.
- `session.use_strict_mode` rifiuta ID di sessione non validi o creati manualmente, così da contrastare attacchi di tipo session fixation, in cui un malintenzionato tenta di imporre un ID di sessione già noto.
- `session.cookie_httponly` e `session.cookie_secure` proteggono il cookie rendendolo inaccessibile via JavaScript e trasmissibile solo su connessioni HTTPS, riducendo i rischi legati a furti o intercettazioni.
- `session.cookie_samesite` con valore `Strict` limita l'invio del cookie a richieste provenienti dallo stesso sito, ostacolando eventuali attacchi CSRF basati su richieste cross-site.

In aggiunta, è stato definito un nome personalizzato per il cookie di sessione mediante l'istruzione `session_name('id')`, prima dell'inizializzazione della sessione. Tale scelta ha l'obiettivo di assegnare all'identificativo un nome generico e meno prevedibile rispetto a quello predefinito utilizzato da PHP, riducendo così la possibilità che un potenziale attaccante possa individuarlo e sfruttarlo.

```php
// Impostazione delle direttive di sicurezza della sessione
ini_set('session.use_only_cookies', 1);
ini_set('session.use_strict_mode', 1);
ini_set('session.cookie_httponly', 1);
ini_set('session.cookie_secure', 1);
```

```php
ini_set('session.cookie_samesite', 'Strict');

// Cambio del nome del cookie di sessione
session_name('id');

//avvio della sessione, se non è già attiva
if( session_status() !== PHP_SESSION_ACTIVE ){
    session_start();
}
```

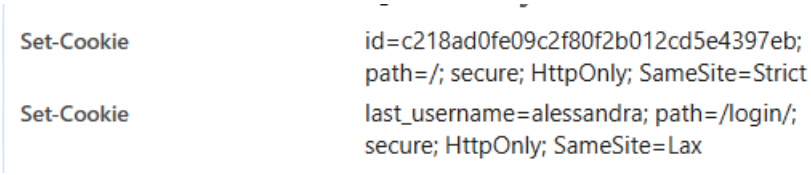


Immagine 4.4: *Cookie di Twittle*

Un ulteriore meccanismo di rafforzamento riguarda la rigenerazione dell'identificativo di sessione tramite `session_regenerate_id(true)`. Questa procedura viene eseguita in corrispondenza di eventi critici, come il cambio di livello di privilegio successivo al login o alla registrazione di un nuovo utente. La rigenerazione dell'ID di sessione consente di invalidare eventuali identificativi precedentemente utilizzati, mitigando in maniera efficace il rischio di attacchi di tipo session fixation.

Infine, al fine di impedire la memorizzazione in cache di dati sensibili da parte del browser o di eventuali proxy intermedi, sono state introdotte specifiche intestazioni HTTP di controllo del caching:

```php
header("Cache-Control: no-store, no-cache, must-revalidate, max-age=0");
header("Cache-Control: post-check=0, pre-check=0", false);
header("Pragma: no-cache");
```

Nello specifico, la direttiva `Cache-Control: no-store, no-cache, must-revalidate, max-age=0` indica al browser di non memorizzare alcuna copia della risorsa, di non utilizzare eventuali versioni memorizzate e di convalidare sempre la risorsa con il server. I parametri `post-check=0` e `pre-check=0` sono stati aggiunti per garantire compatibilità con versioni meno recenti di Internet Explorer, che interpretano queste opzioni per controllare il comportamento di aggiornamento della cache. L'intestazione `Pragma: no-cache` è invece uno standard HTTP/1.0 utilizzato ancora per motivi di retrocompatibilità, con lo stesso obiettivo di disabilitare la memorizzazione locale delle risorse.

Queste intestazioni, assenti nella configurazione originaria, costituiscono un accorgimento fondamentale per garantire la riservatezza delle informazioni veicolate tramite la sessione. In questo modo si riduce la possibilità che pagine o contenuti protetti possano essere recuperati successivamente attraverso la cronologia del browser o da copie residue memorizzate nella cache locale o da proxy intermedi.

4.3.2 Timeout di Sessione

Un'altra importante novità introdotta riguarda l'implementazione di un controllo sistematico sul tempo di vita delle sessioni, realizzato nel file `app/Session/sessionManager.php`. Originariamente, la sessione restava valida indefinitamente, fino a chiusura del browser o a interventi manuali da parte dell'utente, con conseguenti rischi di abuso in caso di dispositivi condivisi o di inattività prolungata. Per ovviare a tale criticità, sono stati definiti due timeout:

- Un timeout di inattività di 15 minuti, che interrompe automaticamente la sessione se non viene rilevata alcuna attività da parte dell'utente.

- Un timeout assoluto di 4 ore, dopo il quale la sessione viene terminata indipendentemente dall'attività svolta.

Al momento dell'autenticazione (login o registrazione), i parametri di timeout vengono salvati all'interno dell'array `$_SESSION`, così da poter essere verificati a ogni interazione successiva.

```
$_SESSION['created'] = time();   // timeout assoluto
$_SESSION['last_activity'] = time(); //timeout di inattività
```

Durante l'esecuzione di ogni richiesta nelle pagine riservate, il sistema verifica costantemente se l'intervallo di inattività o la durata complessiva della sessione abbiano superato i limiti stabiliti. Questi controlli vengono effettuati attraverso l'inclusione del file `sessionManager.php`, inserito subito dopo la verifica dell'autenticazione dell'utente in tutte le sezioni che richiedono l'accesso protetto. Qualora uno dei vincoli temporali venga superato, la sessione viene invalidata in modo sicuro e l'utente viene reindirizzato alla pagina di login, accompagnato da un messaggio che ne spiega la causa. Per assicurare un monitoraggio preciso dell'attività dell'utente, al termine di ogni richiesta valida il valore di last_activity viene aggiornato con il timestamp corrente, permettendo così di calcolare correttamente il timeout di inattività.

```
$idleTimeout = 900;   // timeout di inattività di 15 minuti
$absoluteTimeout = 14400; // timeout assoluto di 4 ore

//timeout di inattività
if(isset($_SESSION['last_activity']) && (time() - $_SESSION['last_activity'] >
$idleTimeout)){
       session_unset();
       session_destroy();
       header("Location: login.php");
       $_SESSION['error_message'] = "La sessione è scaduta per inattività.
       Effettua nuovamente il login.";
       exit();
}

//aggiornamento di last activity
$_SESSION['last_activity'] = time();

//timeout assoluto
if(isset($_SESSION['created']) && (time() - $_SESSION['created'] >
$absoluteTimeout)){
       session_unset();
       session_destroy();
       header("Location: login.php");
       $_SESSION['error_message'] = "La sessione è scaduta. Effettua nuovamente
       il login.";
    exit();
}
```

4.3.3 Gestione dei Cookie

Infine, come misura di comodità per l'utente, in origine era stato introdotto un cookie persistente per la memorizzazione dello username con una durata di 16 ore. Tuttavia, al fine di ridurre i rischi legati alla conservazione prolungata di dati sul client, tale cookie è stato modificato per diventare un cookie di sessione, eliminato automaticamente alla chiusura del browser. Il cookie viene inoltre configurato con opportune restrizioni [14]:

- Il campo Path è limitato esclusivamente alla directory /login/, in modo da circoscrivere la portata del cookie solo all'area di autenticazione, riducendo l'esposizione a parti non necessarie del sito.
- Gli attributi Secure e HttpOnly sono impostati per garantire rispettivamente che il cookie venga trasmesso esclusivamente tramite connessioni HTTPS sicure e che sia inaccessibile tramite JavaScript lato client, mitigando il rischio di furto attraverso attacchi di tipo Cross-Site Scripting (XSS).
- L'attributo SameSite è configurato su Lax, che permette di limitare l'invio del cookie in contesti cross-site solo a determinati metodi di navigazione "sicuri", contribuendo a prevenire attacchi di Cross-Site Request Forgery (CSRF) senza compromettere la funzionalità legittima del sito.
- Non viene specificato alcun valore per il campo Domain, in modo tale che il cookie sia valido esclusivamente per il dominio di origine, evitando la condivisione indesiderata con sottodomini o domini esterni.

```php
// --- login.php ---

//configurazione precedente, priva di ogni parametro di sicurezza
setcookie('last_username', $username, time() + 16 * 3600, '/');

//nuova configurazione
setcookie('last_username', urlencode($username),[
'path' => '/login/',    //ristretto solo all'area login
 'secure' => true,      // invio del cookie solo tramite HTTPS
 'httponly' => true,   // inaccessibile via javascript
 'samesite' => 'Lax', //prevenzione contro CSRF
]);
```

4.4 Mitigazione del Cross-Site Scripting (XSS)

La prevenzione delle vulnerabilità di tipo Cross-Site Scripting (XSS) rappresenta un aspetto cruciale per la sicurezza di qualsiasi applicazione web, poiché permette di contrastare l'iniezione e l'esecuzione di codice JavaScript malevolo da parte di utenti non autorizzati. Tali attacchi possono compromettere la riservatezza dei dati, manipolare l'interfaccia utente o dirottare sessioni autenticate, generando conseguenze rilevanti per la sicurezza complessiva del sistema [15].

Nel contesto di questo progetto, la gestione delle problematiche legate a XSS è stata oggetto di una revisione approfondita del codice originario, che non prevedeva adeguate contromisure. A tal fine, sono stati introdotti interventi mirati e sistematici volti a rafforzare la protezione contro questa tipologia di vulnerabilità, attraverso un approccio multilivello e coerente, applicato in maniera trasversale a tutte le

pagine dell'applicativo. Tale strategia si fonda su una combinazione di tecniche complementari, tra cui figurano:

- *Validazione dell'input*: ogni dato in ingresso viene sottoposto a una fase di validazione, effettuata sia lato client, tramite JavaScript, che lato server, attraverso controlli in PHP. Questo duplice livello di verifica permette di intercettare tempestivamente input anomali, riducendo il rischio che contenuti potenzialmente dannosi raggiungano il backend o vengano processati dal browser.

- *Output encoding lato server*: prima che qualsiasi contenuto dinamico venga inserito nel DOM e visualizzato dall'utente, esso viene sottoposto a una procedura di *output encoding*, utilizzando le funzioni PHP `htmlspecialchars()` o `htmlentities()`. Tali funzioni trasformano i caratteri speciali, come virgolette e simboli di apertura HTML, in entità sicure, evitando l'esecuzione involontaria di codice.

```
// PRIMA dell'output encoding, il contenuto veniva stampato direttamente a video
//senza alcun controllo
echo isset($username) ? $username : '';

//output encoding
echo isset($username) ? htmlspecialchars($username, ENT_QUOTES |
ENT_SUBSTITUTE, 'UTF-8') : '';
```

  L'utilizzo dei flag `ENT_QUOTES` e `ENT_SUBSTITUTE` assicura rispettivamente la codifica sia delle virgolette doppie che singole, e la sostituzione di eventuali caratteri non validi in UTF-8, prevenendo errori di rendering e potenziali exploit.

- *Sanificazione lato client*: per garantire un controllo sull'output HTML generato lato client, specialmente nella pubblicazione e visualizzazione dei "*twit*" nelle pagine `bacheca.php` e `scopri.php`, è stata adottata la libreria `DOMPurify`, una soluzione riconosciuta e consigliata in *OWASP Code Review Guide* per la prevenzione delle vulnerabilità di tipo Cross-Site Scripting (XSS) [16].
  L'impiego di `DOMPurify` si è reso necessario per consentire la pubblicazione di testi arricchiti da alcuni tag HTML considerati sicuri, come quelli per la formattazione (`<b>`, `<i>`, `<u>`, `<strong>`) o per l'inserimento di link (`<a>`), senza dover convertire completamente il contenuto in semplice testo. Questo approccio offre maggiore flessibilità agli utenti, preservando al contempo elevati standard di sicurezza.
  Per garantire che tali contenuti non venissero renderizzati direttamente in modo non controllato, il testo originario viene salvato in un attributo personalizzato, `data-raw`, codificato in base64. In fase di rendering, lo script client-side recupera il valore, lo decodifica e lo passa a `DOMPurify`, il quale filtra il contenuto secondo una lista predefinita di tag e attributi consentiti. In particolare, la configurazione utilizzata consente solo tag HTML notoriamente innocui e utili per la formattazione, oltre agli attributi `href`, `title`, `target` e `rel` per i link.

  Inoltre, è stato implementato un hook di sicurezza aggiuntivo che interviene nella fase di sanificazione degli attributi HTML, con lo scopo di rafforzare ulteriormente la gestione dei link esterni. Questo meccanismo verifica che l'attributo `href` dei tag `<a>` faccia esplicitamente riferimento a URL che utilizzano esclusivamente il protocollo sicuro https. In caso contrario, l'attributo viene rimosso per impedire qualsiasi reindirizzamento verso risorse potenzialmente pericolose. Oltre a ciò, il link viene automaticamente arricchito con le direttive `rel="noopener noreferrer"` e `target="_blank"`. Questa configurazione è fondamentale per evitare attacchi di tipo reverse tabnabbing, in cui la nuova scheda aperta potrebbe tentare di manipolare la pagina di origine attraverso l'oggetto `window.opener`. Impedendo questa relazione tra finestre, si

preserva l'integrità dell'interfaccia utente e si riduce il rischio di escalation di privilegi o spoofing del contenuto.

```
//PRIMA della mitigazione: stampa del contenuto direttamente
echo '<p class="content">'.$tweet['testo']."</p>";


 //DOPO
// Definizione dei tag e attributi HTML consentiti
const allowedTags = ['b', 'i', 'u', 'strong', 'em', 'a'];
const allowedAttrs = ['href', 'title', 'target', 'rel'];

// Hook personalizzato per la sicurezza dei link <a>
DOMPurify.addHook('afterSanitizeAttributes', function (node) {
    if (node.tagName === 'A' && node.hasAttribute('href')) {
        const href = node.getAttribute('href').trim();

        // Rimuove l'attributo href se il protocollo non è https
        if (!/^https:\/\//i.test(href)) {
            node.removeAttribute('href');
        } else {
            // Aggiunta di attributi per prevenire reverse tabnabbing
            node.setAttribute('rel', 'noopener noreferrer');
            node.setAttribute('target', '_blank');
        }
    }
});


//vengono selezionati gli elementi data-raw
document.querySelectorAll('[data-raw]').forEach(el => {
   const encoded = el.getAttribute('data-raw');

    // Decodifica il contenuto base64
    let dirty = '';
    try {
        dirty = atob(encoded);
    } catch (e) {
        dirty = '';
    }

    // sanificazione del contenuto html mantenendo solo i tag consentiti
    const clean = DOMPurify.sanitize(dirty, {
      ALLOWED_TAGS: allowedTags,
      ALLOWED_ATTR: allowedAttrs
    });

    // viene inserito il contenuto sanificato nella pagina
    el.innerHTML = clean;
```

```
        });
```

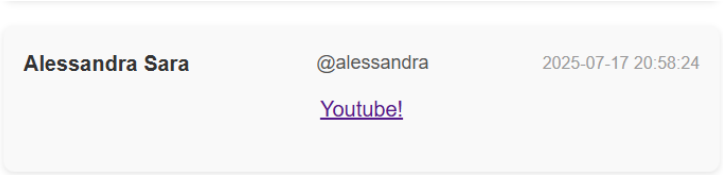


Immagine 4.5: *Creazione di Twit con link sicuro*

- *Eliminazione degli script inline e uso di tecniche sicure*: tutti gli script inline, compresi i gestori di eventi HTML (`onclick`, `onerror`, `onsubmit`, ecc.), sono stati rimossi dalle pagine pubbliche, in conformità con le best practice per la sicurezza client-side. L'intero codice JavaScript è stato rifattorizzato e spostato in un file esterno (common.js), la cui esecuzione è autorizzata tramite un nonce definito nella CSP.
  È stato inoltre evitato l'uso di API JavaScript potenzialmente pericolose come `eval()`, `document.write()` e l'assegnazione diretta a `innerHTML`, preferendo alternative più sicure come `Node.textContent` per la manipolazione di contenuti testuali, e `Element.setAttribute()` per l'impostazione sicura degli attributi.
  Fa unicamente eccezione l'uso mirato di innerHTML in combinazione con DOMPurify, una libreria raccomandata dalla *OWASP Code Review Guide* per la sanificazione lato client. In questo caso, il contenuto HTML viene preventivamente filtrato per rimuovere eventuali elementi o attributi pericolosi prima della sua inserzione nel DOM.

- *Intestazioni di sicurezza http*: all'interno del file `app/Session/init.php`, vengono definite alcune intestazioni HTTP fondamentali per rafforzare la sicurezza lato client, riducendo la superficie d'attacco contro tecniche comuni di exploit basate sul comportamento predefinito del browser. In particolare, sono stati configurati i seguenti header:

```php
// Intestazioni di sicurezza
header("X-Content-Type-Options: nosniff");
header("X-Frame-Options: SAMEORIGIN");  // o DENY
header("Referrer-Policy: no-referrer");
```

  In particolare, l'intestazione `X-Content-Type-Options: nosniff` impedisce al browser di effettuare il MIME sniffing, forzandolo a rispettare il tipo di contenuto dichiarato dal server ed evitando così che file potenzialmente pericolosi vengano interpretati come script. La direttiva `X-Frame-Options: SAMEORIGIN` limita invece l'inclusione della pagina in frame o iframe a contenuti provenienti dallo stesso dominio, prevenendo attacchi di tipo clickjacking. Infine, l'intestazione `Referrer-Policy: no-referrer` disabilita l'invio dell'header Referer, impedendo che informazioni sensibili o URL della pagina di origine vengano trasmessi a siti esterni.

4.5 Mitigazione del Cross-Site Request Forgery (CSRF)

Il Cross-Site Request Forgery (CSRF) rappresenta una vulnerabilità di sicurezza che sfrutta il comportamento predefinito dei browser, i quali allegano automaticamente i cookie di sessione a ogni richiesta HTTP inviata verso il dominio di destinazione. [17] Un attaccante, inducendo un utente già autenticato a visitare una risorsa esterna o a interagire con contenuti appositamente predisposti, può generare richieste indesiderate verso l'applicazione legittima. In questo modo, azioni potenzialmente critiche, come la modifica di dati o l'esecuzione di operazioni privilegiate, possono essere effettuate all'insaputa dell'utente, compromettendo l'integrità e l'affidabilità del sistema.

Nel progetto oggetto di studio, il codice originale non implementava alcuna protezione specifica contro questa vulnerabilità. A tal fine, sono stati introdotti diversi accorgimenti per rafforzare la sicurezza delle operazioni sensibili.

4.5.1 Implementazione del Synchronizer Token Pattern

Per ogni richiesta HTTP di tipo POST, o più in generale per tutte le operazioni in grado di produrre modifiche ai dati lato server, è stato implementato un meccanismo di protezione basato sull'utilizzo di un token CSRF univoco e non prevedibile. Tale token viene generato lato server all'interno del file `app/Session/init.php` e memorizzato in modo sicuro nella variabile di sessione `$_SESSION['csrf_token']`, così da restare legato in modo sicuro alla sessione dell'utente autenticato.

```
// generazione del token CSRF se non esiste
if (empty($_SESSION['csrf_token'])) {
    $_SESSION['csrf_token'] = bin2hex(random_bytes(32));

}
```

Ad ogni invio di dati tramite form HTML, il token viene incluso automaticamente come campo nascosto, così da essere trasmesso insieme ai dati inviati dall'utente.

```
<form id="loginForm" action="login.php" method="post">
      <input type="hidden" name="csrf_token" value="<?php echo
      htmlspecialchars($_SESSION['csrf_token'], ENT_QUOTES | ENT_SUBSTITUTE,
      'UTF-8'); ?>">
      <!-- Altri campi del form -->
</form>
```

Al momento dell'elaborazione della richiesta, il server procede a verificare sia l'esistenza sia la corrispondenza del token CSRF ricevuto rispetto a quello originariamente generato e memorizzato nella variabile di sessione. Qualora non vi sia corrispondenza tra i due valori, o il token risulti del tutto assente, l'operazione viene immediatamente annullata. In tali circostanze, l'applicazione registra un log dell'evento; successivamente, viene riaggiornata la pagina, mostrando un messaggio di errore esplicativo che invita a ripetere l'operazione.

```
if(!isset($_POST['csrf_token']) || !hash_equals($_SESSION['csrf_token'],
$_POST['csrf_token'])){
        $_SESSION['error_message'] = "Richiesta non valida. Riprova.";
        logErrore("CSRF token mismatch");
        header("Location: registrazione.php");
        exit();
    }
```

4.5.2 Impostazione dell'Attributo SameSite per i Cookie

Parallelamente, è stato configurato l'attributo SameSite sui cookie, per limitare l'invio automatico dei cookie di sessione in contesti cross-site.
Nella configurazione attuale, i cookie di sessione utilizzano `SameSite=Strict`, garantendo così che non vengano trasmessi in alcuna richiesta proveniente da domini esterni. Questa scelta riduce drasticamente la superficie di attacco per i tentativi di CSRF:

```
//--- init.php ---
ini_set('session.cookie_samesite', 'Strict');
```

Per i cookie non critici, come quello per la memorizzazione dello username nella pagina di login, viene invece adottata la modalità Lax, che offre un equilibrio tra sicurezza e usabilità:

```
setcookie('last_username', urlencode($username),[
    'path' => '/login/',
     'secure' => true,
     'httponly' => true,
     'samesite' => 'Lax',
]);
```

Questa configurazione consente ai cookie di essere inviati in normali richieste di navigazione (ad esempio clic su link esterni), ma li esclude da metodi HTTP potenzialmente pericolosi come POST o PUT provenienti da siti terzi.

4.5.3 Verifica delle Intestazioni Origin e Referer

Infine, è stata introdotta una verifica aggiuntiva sull'origine delle richieste, attraverso il controllo delle intestazioni HTTP Origin e Referer. Questa misura fornisce una barriera difensiva supplementare nel caso in cui un attaccante tenti di inviare richieste cross-site sfruttando eventuali lacune o cercando di bypassare altre contromisure, come i token CSRF.

Il controllo è stato implementato all'interno del file `/app/check_origin.php`, il quale viene incluso all'inizio di tutte le pagine sensibili, subito prima della verifica del token di sincronizzazione. In termini pratici, lo script recupera le intestazioni Origin e Referer della richiesta HTTP in ingresso e le confronta con un elenco di domini considerati attendibili.

Se almeno una delle intestazioni coincide con uno dei domini autorizzati, la richiesta viene ritenuta lecita e l'elaborazione prosegue regolarmente; in caso contrario, viene registrato un log dell'evento e viene restituito un messaggio di errore HTTP 403 *Forbidden*, interrompendo immediatamente l'esecuzione.

Questo duplice controllo, basato su Origin e Referer, consente di mitigare scenari in cui un header possa essere assente o manipolato, rafforzando così la resilienza dell'applicazione contro tentativi di attacco di tipo Cross-Site Request Forgery (CSRF) o altre richieste cross-origin non autorizzate.

```
$allowedDomain = ['https://195.231.2.153.sslip.io'];

 function checkOriginReferer($allowedDomains){
    // Vengono recuperate le intestazioni
    $origin = $_SERVER['HTTP_ORIGIN'] ?? '';
    $referer = $_SERVER['HTTP_REFERER'] ?? '';

    $isValid = false;

    foreach($allowedDomains as $domain){
        //controllo se origin combacia
        if(!empty($origin) && $origin === $domain ){
            $isValid = true;
            break;
        }

        //successivamente viene confrontato referer
```

```
        if (!empty($referer) && strpos($referer, $domain) === 0) {
            $isValid = true;
            break;
        }
    }

    if(!$isValid){
        //log dell'evento
         LogErrore('Richiesta con Origin/Referer non valido da: ' . ($origin ?:
      $referer));
        header($_SERVER["SERVER_PROTOCOL"] . " 403 Forbidden");
        echo "Richiesta non autorizzata. Origine non valida.";
        exit();
    }
}
```

4.6 Mitigazione delle Security Misconfiguration

La security misconfiguration costituisce una delle cause più frequenti di vulnerabilità nelle applicazioni web e può essere sfruttata da attaccanti per compromettere la riservatezza, l'integrità o la disponibilità dei dati. Questa problematica si verifica quando un sistema non è configurato secondo pratiche di sicurezza appropriate oppure quando impostazioni predefinite, spesso insicure, non vengono adeguatamente modificate durante il deployment dell'applicazione. Errori comuni includono la permanenza di configurazioni di default attive, una gestione non sicura dei file di configurazione o l'assegnazione di permessi eccessivi o non necessari ai componenti del sistema.

Nel progetto sviluppato, sono state adottate misure specifiche per ridurre al minimo questo rischio, agendo direttamente sui parametri di configurazione PHP attraverso il file `app/Session/init.php`. Le principali configurazioni di sicurezza implementate sono descritte di seguito [18]:

- `ini_set('display_errors', '0')`: disabilita la visualizzazione degli errori PHP direttamente nel browser, evitando che dettagli interni come stack trace, percorsi di file o messaggi di errore possano essere esposti a utenti malintenzionati. In questo modo si riduce la possibilità che informazioni potenzialmente sensibili vengano sfruttate per individuare vulnerabilità.

- `ini_set('log_errors', '1')`: abilita la registrazione degli errori, dirottando i messaggi di errore verso un file di log sicuro sul server. Questo permette agli sviluppatori di effettuare analisi puntuali in fase di debugging, senza divulgare informazioni tecniche all'esterno.

- `ini_set('error_log', '/config/workspace/PWR_twittle_TESI/app/log/errori.log')`: definisce il percorso del file di log per la memorizzazione degli errori.

- `ini_set('expose_php', 'Off')`: disabilita l'invio dell'header X-Powered-By nelle risposte HTTP, impedendo che venga rivelata la versione di PHP utilizzata dall'applicazione. Questa misura riduce la superficie di attacco, ostacolando eventuali tentativi di sfruttamento di vulnerabilità note legate a versioni specifiche. È importante sottolineare che l'efficacia completa di questa configurazione richiede l'impostazione diretta a livello di server tramite il file php.ini. Nel presente progetto, tuttavia, non è stato possibile accedere alla configurazione del server né modificare il file php.ini. Inoltre, non è stato possibile applicare questa direttiva tramite il file

`.htaccess`, poiché il modulo `mod_headers` di Apache non è attivo sul server in uso. Per tali motivi, la direttiva è stata inserita tramite `ini_set()` nel codice PHP a scopo esplicativo e documentativo, pur consapevoli che questo metodo potrebbe non garantire il risultato desiderato in tutti gli ambienti di esecuzione.

- L'header HTTP Server, che espone informazioni sul software e la versione del server web, non è stato possibile rimuoverlo o modificarlo in quanto la configurazione richiede accesso diretto al server o al modulo `mod_headers` di Apache, entrambi non disponibili nell'ambiente attuale. Di conseguenza, tale header rimane visibile, rappresentando un potenziale rischio per la sicurezza

- `ini_set('file_uploads', '0')`: disabilita la possibilità di caricare file tramite form HTML. Dal momento che l'applicazione Twittle non prevede funzionalità di upload di file da parte dell'utente, questa opzione contribuisce a minimizzare il rischio di caricamento di contenuti malevoli, come web shell o script non autorizzati.

Oltre a queste misure generali, particolare attenzione è stata posta anche alla configurazione delle sessioni, già approfondita però nei paragrafi precedenti.

4.7 Mitigazione del Rischio di Sensitive Data Exposure

L'esposizione di dati sensibili (*Sensitive Data Exposure*) rappresenta una delle categorie di vulnerabilità più gravi per le applicazioni web, in quanto riguarda la compromissione di informazioni riservate come credenziali di accesso, dati personali o sessioni utente [19]. Una gestione inadeguata di questi dati, sia durante la trasmissione (*in transit*) sia durante l'archiviazione (*at rest*), può avere conseguenze rilevanti in termini di perdita di riservatezza, integrità e fiducia da parte degli utenti.

La protezione dei dati *in transito* si fonda sull'obiettivo di prevenire l'intercettazione o la manomissione di informazioni sensibili mentre vengono trasferite tra client e server. A tale scopo sono state implementate diverse misure di sicurezza.

4.7.1 Transport Layer Security (TLS)

L'applicazione Twittle adotta il protocollo TLS (*Transport Layer Security*) per cifrare tutte le comunicazioni HTTP, garantendo così un canale sicuro tra client e server. A tal fine, è stato installato un certificato di tipo Domain Validated (DV) rilasciato da Let's Encrypt, con una validità di 90 giorni e una firma digitale basata sull'algoritmo SHA-384, che assicura un elevato livello di sicurezza crittografica.

L'utilizzo di TLS garantisce diversi aspetti fondamentali della sicurezza delle comunicazioni:

- Cifratura end-to-end: tutti i dati trasmessi tra client e server sono criptati, proteggendoli da eventuali intercettazioni o captazioni da parte di attori non autorizzati.

- Autenticazione del server: il certificato consente al client di verificare l'identità del server, assicurando che la connessione sia stabilita con l'host legittimo e non con un server malevolo che potrebbe intercettare o manipolare i dati.

- Integrità dei dati: TLS garantisce che i dati ricevuti non siano stati alterati o manipolati durante il trasferimento, preservando la correttezza delle informazioni trasmesse.

L'impiego di certificati validi, aggiornati regolarmente e basati su algoritmi crittografici robusti rappresenta un prerequisito essenziale per assicurare la riservatezza, l'affidabilità e la sicurezza delle comunicazioni all'interno di un'applicazione web come Twittle. Tale implementazione costituisce una

componente chiave nel sistema di protezione dei dati in transito, prevenendo attacchi quali intercettazioni (sniffing), manomissioni (tampering) e attacchi di tipo man-in-the-middle.

4.7.2 Reindirizzamento HTTPS

Per garantire che tutte le risorse dell'applicazione siano accessibili esclusivamente tramite connessioni cifrate, è stato configurato un meccanismo di reindirizzamento automatico da HTTP a HTTPS a livello di web server. Tale configurazione, descritta in dettaglio nel capitolo 4.1.1, è implementata tramite regole di riscrittura nel file `.htaccess`. Questo sistema verifica se la richiesta non è già protetta da HTTPS e, in tal caso, effettua un redirect verso l'URL sicuro corrispondente. Di conseguenza, tutte le richieste originariamente inviate in chiaro vengono forzatamente convertite in connessioni cifrate, riducendo significativamente il rischio di intercettazioni o attacchi di tipo man-in-the-middle.

4.7.3 HTTP Strict Transport Security (HSTS)

Per ridurre attacchi di downgrade, come lo SSL stripping, è stata implementata l'intestazione HTTP Strict Transport Security (HSTS). Questa direttiva indica ai browser che il dominio deve essere sempre raggiunto tramite connessioni HTTPS per un periodo di tempo definito, impedendo così l'accesso tramite connessioni non cifrate http [20].

L'intestazione viene inviata dal server solo quando la richiesta è già stabilita su una connessione sicura HTTPS, evitando di forzare HTTPS su richieste in chiaro. In questo modo, il browser riceve l'istruzione di utilizzare esclusivamente HTTPS per tutte le visite future al dominio, rafforzando la protezione contro intercettazioni e modifiche del traffico e migliorando la sicurezza complessiva dell'applicazione web.

```
//--- init.php ---
//intestazione HSTS solo se la pagina è su https
if (isset($_SERVER['HTTPS']) && $_SERVER['HTTPS'] === 'on') {
    header("Strict-Transport-Security: max-age=31536000; includeSubDomains;
preload");
}
```

Con questa configurazione, la policy HSTS è valida per un anno, include tutti i sottodomini e può essere pre-caricata nei browser che supportano l'HSTS Preload List, un elenco mantenuto dai browser per forzare l'uso di HTTPS sin dal primo contatto.

4.7.4 Flag Secure e HttpOnly per i Cookie

Un'ulteriore misura di protezione riguarda i cookie di sessione e quelli personalizzati. Il flag Secure garantisce che i cookie siano trasmessi solo tramite connessioni HTTPS, evitando intercettazioni su canali non cifrati. Il flag HttpOnly, invece, impedisce l'accesso ai cookie da parte di script lato client, riducendo così il rischio di furto di sessione tramite attacchi XSS.

Per i cookie di sessione questi attributi vengono configurati globalmente, mentre per i cookie personalizzati sono impostati in modo esplicito. Questi aspetti sono già stati ampiamente descritti nel capitolo 4.2 – Session Management.

4.7.5 Prevenzione del Caching di Dati Sensibili

Per evitare che informazioni riservate vengano memorizzate nella cache del browser o di eventuali proxy intermedi, vengono impostati intestazioni HTTP specifiche che disabilitano la memorizzazione locale dei dati.

Questa strategia riduce sensibilmente la possibilità che dati sensibili rimangano accessibili anche dopo la chiusura della sessione da parte dell'utente o di terzi non autorizzati. Anche questa misura, insieme alla gestione complessiva della sessione, è stata già ampiamente descritta nel capitolo 4.2– Session Management.

4.7.6 Protezione dei Dati durante l'Archiviazione (*at rest*)

La protezione dei dati at rest comprende l'insieme delle misure adottate per garantire la sicurezza delle informazioni memorizzate in modo persistente, quali dati conservati in database o file di configurazione. L'obiettivo principale è prevenire accessi non autorizzati, furti o manipolazioni dei dati, anche in scenari di compromissione dell'infrastruttura.

Un aspetto particolarmente critico riguarda la gestione sicura delle credenziali utente. In precedenza, infatti, le password venivano archiviate in chiaro, esponendo l'intero sistema a gravi rischi di compromissione di sicurezza in caso di accesso non autorizzato al database.

Al fine di migliorare significativamente la sicurezza, è stato implementato un processo di hashing robusto basato sulla funzione `password_hash()` di PHP con l'algoritmo Argon2id. Questo algoritmo, vincitore del *Password Hashing Competition*, rappresenta uno standard affidabile e aggiornato per la protezione delle credenziali, in grado di resistere efficacemente a tentativi di attacco basati su forza bruta o tecniche di precomputazione come le rainbow table. Di seguito viene riportato un esempio di implementazione:

```
//PRIMA (senza hash delle password), salvato direttamente nel db
$password = $_POST['password'];

// DOPO; Recupero della password in chiaro dall'input utente
$password = $_POST['password'];

// Generazione dell'hash sicuro con Argon2id
$hash = password_hash($password, PASSWORD_ARGON2ID);

// Salvataggio dell'hash nel database
```

, salLa funzione `password_hash()` gestisce automaticamente la generazione di un salt casuale e sicuro per ogni singola password, garantendo che anche password identiche producano hash univoci. La verifica dell'autenticità della password in fase di login viene effettuata in modo sicuro tramite la funzione `password_verify()`:

```
// PRIMA
if ($password === $row['pwd']){
}

// DOPO; Verifica della password fornita dall'utente
if (password_verify($pepperedPassword, $hash_from_db)) {
    // Password corretta
} else {
    // Password errata
}
```

Per incrementare la sicurezza dell'hashing delle password, è stato introdotto un pepper segreto, implementato come chiave aggiuntiva di derivazione crittografica. A differenza del salt, che viene

generato e memorizzato insieme all'hash di ciascuna password, il pepper viene conservato separatamente dal database e dal codice applicativo, in un file di configurazione `.env`.

Il valore del pepper è stato generato come stringa casuale sufficientemente lunga per garantire un livello di entropia adeguato. Questa stringa è stata successivamente inserita all'interno del file `app/security/.env`. Per gestire in modo sicuro il caricamento del pepper, è stato sviluppato lo script `load_env.php`, il cui scopo principale è quello di estrarre e rendere disponibili le variabili di ambiente definite nel file `.env`. Questo approccio consente di separare le informazioni sensibili dal codice sorgente, migliorando la sicurezza complessiva dell'applicazione.

Lo script opera attraverso una funzione dedicata, `loadEnv()`, che esegue le seguenti operazioni:

- Lettura del file `.env`: la funzione verifica innanzitutto l'esistenza del file nel percorso specificato. In caso di assenza, registra l'errore tramite il sistema di logging e interrompe l'esecuzione per evitare che l'applicazione prosegua senza la configurazione necessaria.

- Parsing delle variabili: il contenuto del file viene letto riga per riga, ignorando eventuali commenti o linee vuote. Ogni riga viene quindi suddivisa in coppie chiave-valore, seguendo la sintassi NOME_VARIABILE=VALORE. Queste coppie vengono inserite nell'array globale `$_ENV`, rendendole accessibili all'intera applicazione.

- Validazione del pepper: dopo il caricamento, lo script verifica la presenza della variabile PEPPER_SECRET all'interno di $_ENV. Se questa variabile risulta assente o nulla, viene nuovamente registrato l'errore e l'esecuzione si interrompe, impedendo che il sistema operi senza la chiave segreta fondamentale per la sicurezza delle password.

L'utilizzo di `load_env.php` permette quindi di mantenere il pepper isolato dal codice e dal database, riducendo i rischi di esposizione accidentale o di compromissione in caso di accesso non autorizzato al repository del codice sorgente. Inoltre, grazie a questa separazione, la gestione del pepper diventa più flessibile e modulare: la modifica della chiave segreta avviene esclusivamente a livello di configurazione, senza necessità di intervenire sul codice applicativo.

```php
include_once __DIR__ . '/../log/function_log.php';

function loadEnv($path) {
    if (!file_exists($path)) {
        LogErrore(".env file not found");
        exit();
    }
    $lines = file($path, FILE_IGNORE_NEW_LINES | FILE_SKIP_EMPTY_LINES);
    foreach ($lines as $line) {
        if (strpos(trim($line), '#') === 0) continue; // salta commenti
        list($name, $value) = explode('=', $line, 2);
        $_ENV[trim($name)] = trim($value);
    }
}

loadEnv(__DIR__ . '/.env');

$pepper = $_ENV['PEPPER_SECRET'] ?? null;

if ($pepper === null) {
    LogErrore("Pepper not found in environment variables");
```

```
    exit();
}
```

Infine, lo script è progettato per essere incluso nei moduli di login e registrazione, garantendo che il pepper venga caricato e applicato in modo coerente durante la generazione e la verifica degli hash delle password. Questo approccio contribuisce a rafforzare la resistenza dell'applicazione contro attacchi offline, come quelli di tipo brute-force o rainbow table, assicurando un livello elevato di protezione delle credenziali utente.

```
// --- registrazione.php ----
include_once __DIR__ . "/../app/security/load_env.php";

$peppered = hash_hmac('sha256', $password, $pepper);

//hash della password tramite Argon2id
$hashedPassword = password_hash($peppered, PASSWORD_ARGON2ID);
```

4.8 Mitigazione del Rischio IDOR (Insecure Direct Object Reference)

Nel sistema attuale, l'applicazione consente agli utenti autenticati esclusivamente di creare e leggere i propri tweet. In questo scenario, il rischio di vulnerabilità di tipo IDOR (*Insecure Direct Object References*) è intrinsecamente limitato, in quanto non sono ancora presenti funzionalità che permettano agli utenti di modificare o cancellare risorse [21]. Tuttavia, è fondamentale prevedere sin dalle prime fasi di sviluppo del sistema un insieme di buone pratiche di sicurezza che permettano di mitigare questo rischio anche in caso di future estensioni dell'applicazione.

4.8.1 Filtraggio delle Risorse per Utente

Una prima misura di prevenzione contro attacchi di tipo IDOR consiste nell'assicurare che ogni operazione di accesso ai dati avvenga esclusivamente nel contesto dell'utente autenticato. A tal fine, tutte le query dirette verso il database devono filtrare i dati sulla base dell'identità dell'utente che ha effettuato l'autenticazione.

È buona prassi che l'identificativo dell'utente venga recuperato unicamente dalla sessione di autenticazione gestita server-side, evitando di utilizzare parametri forniti dall'utente stesso. Ciò impedisce a un eventuale attaccante di manipolare l'identificativo per tentare di accedere a risorse altrui.

```
// --- bacheca.php ---
$username = $_SESSION['username'];
```

In questo caso, la variabile `$username` viene valorizzata esclusivamente attraverso la sessione attiva sul server. In questo modo, qualsiasi operazione di lettura o scrittura sui dati risulta vincolata all'identità dell'utente attualmente autenticato, riducendo significativamente il rischio di accesso non autorizzato.

4.8.2 Gestione Sicura degli Identificatori Univoci

Qualora, in un'evoluzione futura, l'applicazione introducesse operazioni di aggiornamento o cancellazione di risorse sarebbe necessario porre particolare attenzione alla gestione sicura degli identificatori univoci.

Ogni risorsa dovrebbe essere identificata in modo inequivocabile mediante un campo dedicato, come un ID numerico autoincrementale o un UUID (*Universally Unique Identifier*). L'ID numerico è utile per l'efficienza delle operazioni interne al database grazie alla sua compattezza in termini di storage e indicizzazione. Tuttavia, se utilizzato come identificativo esposto verso l'esterno, l'ID numerico può essere soggetto ad attacchi di enumerazione. Un attaccante potrebbe, infatti, tentare di accedere a risorse incrementando progressivamente l'identificativo (ad esempio id=1, id=2, id=3 e così via).

Per ridurre tale rischio, è consigliabile affiancare all'ID numerico un UUID. Quest'ultimo è generato casualmente e risulta di fatto impraticabile da indovinare per un attaccante privo di accesso alle informazioni interne del sistema. Nell'interfaccia utente o negli URL, sarà quindi preferibile trasmettere l'UUID piuttosto che l'ID numerico. Un esempio può essere illustrato dal seguente frammento HTML:

```
<form action="/update_profile" method="post">
  <!-- Altri campi come nome, email, ecc. -->
  <input type="hidden" name="user_id" value="550e8400-e29b-41d4-a716-
446655440000">
  <button type="submit">Aggiorna Profilo</button>
</form>
```

In questo caso, l'identificatore dell'utente viene trasmesso come UUID all'interno di un campo nascosto del form. Anche in presenza di un identificativo difficile da indovinare, rimane tuttavia essenziale implementare un controllo di proprietà a livello server-side. Dopo aver ricevuto la richiesta, il backend deve verificare che la risorsa oggetto dell'operazione appartenga effettivamente all'utente autenticato. Tale verifica deve avvenire confrontando l'identificativo dell'utente presente nella sessione con quello associato alla risorsa nel database.

4.8.3 Considerazioni Pratiche

Nella pratica professionale, l'approccio più robusto prevede l'uso combinato di un ID numerico autoincrementale e di un UUID. L'ID numerico garantisce prestazioni ottimali per le operazioni interne, mentre l'UUID viene utilizzato per scopi di esposizione esterna, rendendo più difficile l'enumerazione delle risorse.

È fondamentale sottolineare che l'utilizzo di un UUID non sostituisce in alcun modo i controlli di autorizzazione. Qualora l'applicazione non verifichi la reale proprietà della risorsa da parte dell'utente autenticato, un attaccante potrebbe comunque tentare di ottenere un UUID tramite altre vulnerabilità (ad esempio leakage di log, messaggi di errore non adeguatamente gestiti o configurazioni di debug non sicure).

In conclusione, una corretta gestione del rischio IDOR si fonda sull'applicazione coerente di tre principi: filtraggio rigoroso delle risorse per utente, utilizzo sicuro di identificatori univoci e verifica server-side della proprietà delle risorse. Tali misure costituiscono una base solida per garantire un livello di sicurezza adeguato anche in contesti di evoluzione funzionale del sistema.

4.9 Mitigazione dei Redirects e Forwards non Validati

Le vulnerabilità di tipo *unvalidated redirects and forwards* si verificano quando un'applicazione web consente di reindirizzare o inoltrare l'utente verso un'altra risorsa sulla base di input non adeguatamente controllato. Un attaccante può sfruttare questa debolezza per deviare l'utente verso siti di phishing,

indurre il download di malware o accedere a file interni dell'applicazione, esponendolo così a furti di dati o compromissioni del sistema [21].

4.9.1 Redirect non validati

Un redirect non validato si verifica quando la destinazione di un reindirizzamento HTTP è determinata da un parametro fornito dall'utente, senza alcun controllo lato server. Un esempio classico in PHP può essere rappresentato dalla seguente istruzione:

```
header("Location: " . $_GET['url']);
```

In uno scenario di questo tipo, un attaccante potrebbe manipolare il parametro url per forzare un utente a visitare un sito esterno malevolo, come una pagina di phishing, inducendolo così a divulgare credenziali o altre informazioni riservate.

Nel caso specifico di Twittle, tale rischio risulta attualmente mitigato: l'applicazione non presenta redirect non validati, poiché le destinazioni dei reindirizzamenti vengono definite esclusivamente lato server. In questo modo, l'utente non può in alcun modo influenzare la destinazione finale di un redirect, garantendo un livello di sicurezza più elevato contro questo tipo di attacco.

Qualora in futuro vengano introdotti meccanismi di redirect basati su parametri di input, sarà fondamentale adottare buone pratiche di validazione. In particolare, è consigliabile:

- Evitare di utilizzare URL esterni forniti dall'utente.
- Definire una *whitelist* di destinazioni consentite, gestita lato server.
- Prevedere sempre una destinazione di fallback sicura se l'input non è valido.

Un esempio di implementazione sicura può essere il seguente:

```
// Elenco delle destinazioni consentite
$allowed_urls = [
    'home' => '/home.php',
    'profile' => '/user/profile.php'
];

// Recupera parametro, fallback su 'home'
$page = $_GET['page'] ?? 'home';

// Verifica che la pagina sia consentita
if (isset($allowed_urls[$page])) {
    header('Location: ' . $allowed_urls[$page]);
    exit;
} else {
    // Destinazione di fallback sicura
    header('Location: /home.php');
    exit;
}
```

In questo modo l'applicazione limita le destinazioni di redirect alle sole pagine definite come valide, impedendo reindirizzamenti arbitrari verso siti esterni.

4.9.2 Forward non validati

Diversamente dai redirect, i forward non validati (o *unvalidated forwards*) sono operazioni interne in cui l'applicazione inoltra la gestione della richiesta a un'altra risorsa server-side. In PHP non esiste una funzionalità di *forward* nativa come in alcuni framework Java (ad esempio RequestDispatcher.forward()), ma tale comportamento viene spesso simulato mediante costrutti come `include` o `require`.

Un esempio di forward non validato è illustrato nel frammento seguente:

```
$page = $_GET['page'];

include($page . '.php');
```

In una configurazione simile, un attaccante può manipolare il parametro page per tentare di includere file arbitrari, con rischi di *Directory Traversal* o *Local File Inclusion (LFI)*. Ciò potrebbe consentire la lettura di file sensibili presenti sul server o l'esecuzione di codice malevolo.

Nel caso specifico di Twittle, questa tipologia di vulnerabilità non si presenta, in quanto l'applicazione non implementa alcuna logica di forward basata su input dinamici dell'utente. Tutte le risorse incluse lato server sono definite staticamente nel codice, escludendo la possibilità di eseguire inclusioni di file arbitrari.

Nel caso in cui in futuro venissero introdotti meccanismi di inclusione dinamica, sarebbe comunque fondamentale proteggere questi punti applicando controlli rigorosi. Le buone pratiche raccomandano di:

- Definire una lista bianca (*whitelist*) di file PHP autorizzati.
- Utilizzare percorsi sicuri sfruttando costanti come `__DIR__` per evitare attacchi di traversal.
- Prevedere sempre un file di fallback nel caso l'input non risulti valido.

Un esempio di implementazione sicura è riportato di seguito:

```
$username = $_SESSION['username'];

$allowed_pages = ['home', 'profile', 'dashboard'];

// Recupera parametro, fallback su 'home'
$page = $_GET['page'] ?? 'home';

// Verifica che la pagina sia autorizzata
if (in_array($page, $allowed_pages, true)) {
    include __DIR__ . '/pages/' . $page . '.php';
} else {
    include __DIR__ . '/pages/home.php';
}
```

In questo caso, la whitelist limita l'inclusione a file noti e autorizzati, mentre l'utilizzo di __DIR__ confina fisicamente i file inclusi all'interno di una directory specifica. Questo approccio impedisce eventuali manipolazioni dei percorsi e protegge l'applicazione da accessi non autorizzati a file di sistema.

Capitolo 5

Analisi post-mitigazione e Risultati

5.1 Introduzione

Nei capitoli precedenti sono state analizzate le vulnerabilità presenti nell'applicazione Twittle, emerse durante una prima scansione condotta con OWASP ZAP, e sono state successivamente implementate una serie di contromisure mirate alla loro mitigazione. Tali interventi, descritti nel dettaglio nel Capitolo 4, hanno riguardato sia il codice dell'applicazione che alcuni aspetti della configurazione del server, con l'obiettivo di ridurre significativamente il livello di rischio complessivo.

Per completare questo percorso, è stato fondamentale procedere con una fase di verifica, finalizzata a valutare l'efficacia reale delle azioni correttive intraprese. Questa attività di controllo, svolta attraverso una seconda scansione dinamica nelle stesse condizioni della precedente, ha permesso di osservare l'effettivo impatto degli interventi e di determinare se le vulnerabilità rilevate in precedenza fossero state effettivamente rimosse.

In un processo strutturato di sicurezza applicativa, la fase di validazione riveste un ruolo fondamentale, poiché permette di verificare in modo concreto l'efficacia degli interventi adottati. L'introduzione di contromisure, per quanto tecnicamente corretta, non può considerarsi conclusiva senza una verifica puntuale del loro impatto. È infatti necessario accertarsi che le vulnerabilità siano state effettivamente risolte e che le modifiche apportate non abbiano generato nuovi rischi o compromesso la stabilità complessiva dell'applicazione.

Il presente capitolo è dedicato all'analisi di questa fase conclusiva del processo: ne vengono presentati gli obiettivi, la metodologia adottata e i risultati ottenuti, con un confronto puntuale rispetto alla prima scansione, al fine di valutare in modo chiaro l'efficacia complessiva delle contromisure implementate.

5.2 Obiettivi della verifica post-mitigazione

La verifica post-mitigazione è stata condotta con l'obiettivo di valutare in maniera concreta l'efficacia delle contromisure implementate a seguito della prima scansione. In primo luogo, si è voluto accertare l'eliminazione delle vulnerabilità precedentemente individuate, con particolare attenzione a quelle classificate a rischio alto e medio, considerate prioritarie nell'intervento correttivo.

Parallelamente, la scansione ha permesso di validare la correttezza delle modifiche applicate al codice e alla configurazione del server, verificando che non fossero state introdotte nuove problematiche o comportamenti inattesi. Questo aspetto è stato ritenuto fondamentale per garantire che il processo di mitigazione non avesse effetti collaterali indesiderati sul piano della sicurezza.

Infine, la fase di verifica ha avuto anche uno scopo metodologico, fornendo un riscontro oggettivo sull'approccio seguito e offrendo l'opportunità di riflettere sui punti di forza e sulle eventuali criticità emerse nel corso del processo. In questo senso, la scansione post-mitigazione ha rappresentato il momento conclusivo e di sintesi di un ciclo strutturato di analisi, intervento e controllo.

5.3 Metodologia della scansione

La scansione post-mitigazione è stata condotta utilizzando nuovamente lo strumento OWASP ZAP, configurato come proxy locale per intercettare e analizzare il traffico tra browser e applicazione. L'ambiente operativo, le impostazioni di configurazione e il profilo di scansione sono rimasti invariati rispetto a quanto descritto nel Capitolo 3.2, questa scelta ha permesso di garantire la piena ripetibilità del test e la confrontabilità dei risultati, evitando che eventuali variazioni dipendessero da fattori strumentali anziché da reali cambiamenti nel profilo di rischio dell'applicazione.

Anche in questa fase è stato utilizzato OWASP ZAP nella versione 2.16.1, con il profilo Developer Full, in grado di coprire un'ampia gamma di vulnerabilità comuni nelle applicazioni web. L'attività di scansione si è sviluppata in due momenti distinti: inizialmente si è proceduto con un'esplorazione manuale dell'applicazione, effettuata tramite interazione diretta con l'interfaccia e autenticazione manuale dell'utente. Questo passaggio ha consentito allo scanner di rilevare e memorizzare correttamente tutte le risorse accessibili in sessione autenticata, replicando fedelmente le condizioni operative della prima analisi.

Successivamente, è stata avviata la scansione attiva, in cui lo strumento ha inviato richieste mirate per individuare eventuali vulnerabilità residue. L'intera attività si è svolta in un ambiente controllato, identico a quello utilizzato nella fase iniziale, così da assicurare l'affidabilità dei risultati e l'integrità del confronto tra le due analisi.

5.4 Risultati della Scansione Post-Mitigazione

La scansione finale, effettuata al termine del processo di implementazione delle contromisure correttive, ha restituito un quadro significativamente migliorato e rassicurante sotto il profilo della sicurezza applicativa. In particolare, l'analisi ha evidenziato l'assenza completa di vulnerabilità critiche ad alto e medio rischio, rappresentando un risultato particolarmente significativo che testimonia la solidità raggiunta dal sistema rispetto agli attacchi automatizzati comunemente condotti con strumenti specializzati come OWASP ZAP. Questo esito dimostra come l'implementazione delle contromisure abbia portato a un innalzamento sostanziale del livello di sicurezza dell'applicazione, riducendo drasticamente la superficie di attacco esposta.

Nome	Livello di Rischio	Number of Instances
Server Leaks Information via "X-Powered-By" HTTP Response Header Field(s)	Basso	11
Server Leaks Version Information via "Server" HTTP Response Header Field	Basso	15
Strict-Transport-Security Header Not Set	Basso	4
X-Content-Type-Options Header Missing	Basso	4
Authentication Request Identified	Informativo	2
Cookie Poisoning	Informativo	1
Information Disclosure - Suspicious Comments	Informativo	1
Modern Web Application	Informativo	9
Session Management Response Identified	Informativo	5
User Controllable HTML Element Attribute (Potential XSS)	Informativo	1

Immagine 4.1: *Elenco avvisi di sicurezza*

Nel dettaglio, il report finale generato da OWASP ZAP ha evidenziato la seguente distribuzione delle segnalazioni:

- 0 vulnerabilità ad alto rischio: eliminazione completa delle criticità più severe
- 0 vulnerabilità a rischio medio: assenza di problematiche di sicurezza significative
- 4 avvisi a basso rischio: presenza di elementi minori che richiedono attenzione
- 6 avvisi informativi: segnalazioni di carattere prevalentemente documentale

Questa distribuzione dei risultati dimostra inequivocabilmente come l'applicazione Twittle, a seguito dell'attività di *hardening* sistematico, presenti esclusivamente marginali criticità residue, peraltro riconducibili prevalentemente a best practice non completamente implementate, piuttosto che a vere e proprie vulnerabilità tecniche direttamente sfruttabili da un attaccante.

La totale eliminazione delle vulnerabilità di severità più elevata costituisce un traguardo importante, considerando che tali criticità rappresentano spesso i vettori di attacco primari sfruttati dai malintenzionati per compromettere la sicurezza delle applicazioni web. Il raggiungimento di questo obiettivo indica che l'applicazione ha ora raggiunto un livello di robustezza tale da resistere efficacemente alle tipologie di attacco più diffuse e pericolose nel panorama delle minacce informatiche contemporanee.

Gli avvisi a basso rischio e informativi rimanenti, pur non costituendo vulnerabilità critiche, forniscono indicazioni preziose per ulteriori raffinamenti del sistema e rappresentano opportunità di miglioramento continuo del *posture* di sicurezza dell'applicazione.

5.5 Analisi Tecnica delle Vulnerabilità a Basso Rischio (Gialle)

5.5.1 Esposizione di Informazioni Tecniche

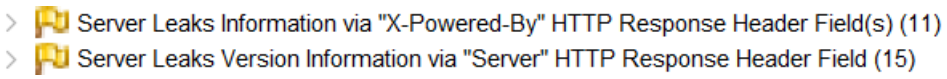


Immagine 4.2: *Esposizione di informazioni tecniche*

- Header esposti: `X-Powered-By`, `Server`
- Gravità: Bassa. Rischio di information disclosure utile per attacchi mirati

Durante la scansione attiva effettuata con OWASP ZAP, sono state rilevate intestazioni HTTP che espongono dettagli tecnici sull'infrastruttura server. In particolare, le risposte includono:

```
X-Powered-By: PHP/7.4.3
Server: nginx/1.15.12
```

Queste informazioni, seppur non critiche, possono facilitare la fase di ricognizione da parte di un attaccante, il quale può sfruttarle per individuare vulnerabilità note (CVE - *Common Vulnerabilities and Exposures*) associate a versioni specifiche di PHP o del web server in uso. L'esposizione del software sottostante rende infatti l'applicazione facilmente aggredibile in caso di configurazioni obsolete o non aggiornate.

Per limitare l'esposizione informativa, è stato tentato di rimuovere le intestazioni `X-Powered-By` e Server tramite il file `.htaccess` e la funzione `ini_set()` di PHP. Tuttavia, nessuno dei due metodi si è rivelato efficace.

Questa inefficacia è probabilmente dovuta a diversi fattori tecnici: la possibile disattivazione del modulo `mod_headers` di Apache, la presenza di un reverse proxy Nginx che potrebbe sovrascrivere gli header in uscita, e restrizioni del servizio di hosting che impediscono modifiche ai file di configurazione globale, come php.ini.

Per questi motivi, pur consapevole delle implicazioni di sicurezza, non è stato possibile eliminare le intestazioni direttamente dal livello applicativo, lasciando tale miglioramento come semplice raccomandazione teorica.

5.5.2 Assenza Parziale degli Header di Sicurezza HSTS e X-Content-Type-Options

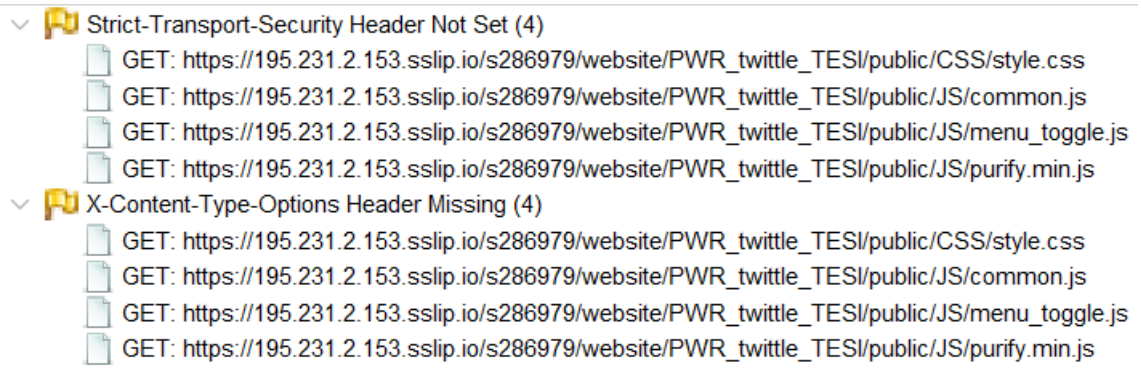


Immagine 4.3: *Elenco avvisi di sicurezza*

- Gravità: Bassa
- Tipo di avviso: Header HTTP mancante su risorse statiche

Durante la fase di scansione post-mitigazione, OWASP ZAP ha rilevato l'assenza degli `header Strict-Transport-Security` (HSTS) e `X-Content-Type-Options: nosniff` in alcune risposte HTTP provenienti dall'applicazione. Nello specifico, l'intestazione HSTS risulta correttamente presente nelle risposte delle principali pagine PHP, come la homepage o le pagine di login, ma non viene

applicata alle risorse statiche, in particolare ai fogli di stile (.css) e agli script JavaScript (.js). Lo stesso comportamento è stato osservato per l'header X-Content-Type-Options, che risulta presente nelle pagine PHP ma assente su file statici.

Entrambi gli header rappresentano misure di sicurezza preventive ampiamente raccomandate: HSTS ha la funzione di forzare l'uso del protocollo HTTPS anche in caso di richieste inizialmente indirizzate a HTTP, mentre X-Content-Type-Options impedisce ai browser di eseguire il cosiddetto *MIME sniffing*, evitando che contenuti serviti con un tipo MIME ambiguo vengano interpretati in modo improprio.

Nel contesto specifico del progetto Twittle, entrambi gli header sono stati implementati correttamente tramite l'utilizzo di uno script PHP dedicato, ovvero il file `app/Session/init.php`. Questo file viene incluso in tutte le pagine PHP dinamiche del sito, assicurando che ogni risposta generata dal motore PHP contenga automaticamente questi header di sicurezza. Tuttavia, una limitazione emersa durante l'analisi effettuata con strumenti automatici come OWASP ZAP riguarda le risorse statiche (immagini, file CSS, JavaScript, ecc.), che non vengono elaborate direttamente dal motore PHP. Infatti, tali risorse vengono solitamente servite in modo diretto dal web server, bypassando completamente l'esecuzione degli script PHP. Di conseguenza, gli header definiti dinamicamente tramite PHP non vengono applicati alle risorse statiche, generando così segnalazioni durante le scansioni automatizzate.

È importante chiarire che questo comportamento non rappresenta una vulnerabilità sfruttabile o una criticità immediata in termini di sicurezza operativa, quanto piuttosto una limitazione intrinseca della configurazione applicativa adottata. Si tratta, nello specifico, di una tipica condizione che si verifica frequentemente negli ambienti di hosting condiviso o in altri scenari in cui il controllo diretto sulla configurazione del web server risulta limitato.

Proprio per questo motivo, in ambienti di produzione dove si possiede il pieno controllo sulla configurazione del server web (come Apache o Nginx), è fortemente consigliabile definire questi header direttamente nel file di configurazione del server. Questa pratica garantisce che tutte le risposte HTTP, incluse quelle relative ai contenuti statici, includano uniformemente le misure di sicurezza necessarie, eliminando così ogni imperfezione rilevabile dalle analisi automatizzate e incrementando complessivamente il livello di sicurezza del sito.

5.6 Analisi Tecnica delle Informazioni (Blu)

5.6.1 Informazioni Potenzialmente Esposte nel Codice e nella Gestione della Sessione

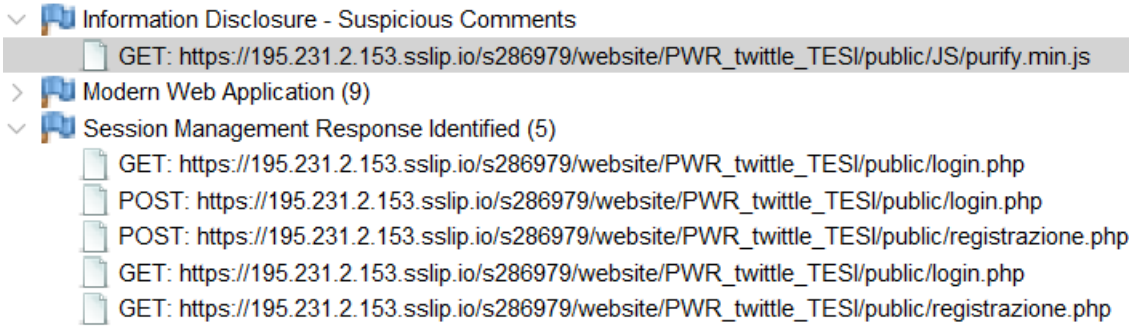


Immagine 4.4: *Avviso di esposizione di informazioni*

- Gravità: Informativo

- Tipo di avviso: Disclosure passiva di informazioni tecniche

OWASP ZAP ha generato due segnalazioni informative relative all'esposizione passiva di informazioni, che, pur non rappresentando vulnerabilità sfruttabili, forniscono indicazioni utili in ottica di miglioramento. La prima riguarda un commento generico rilevato all'interno di `purify.min.js`, una libreria JavaScript di terze parti, che non contiene informazioni sensibili ma evidenzia l'importanza di curare anche il codice importato da fonti esterne.

La seconda segnalazione è relativa alla gestione della sessione utente e viene riportata con l'etichetta "*Session Management Response Identified*". Nonostante il nome del cookie sia stato reso più generico, ZAP ne ha riconosciuto comunque la funzione sulla base del comportamento osservato. Entrambi gli avvisi non richiedono modifiche, ma rappresentano spunti utili per un'ulteriore perfezione della sicurezza applicativa.

5.6.2 Possibile Vulnerabilità di Cookie Poisoning

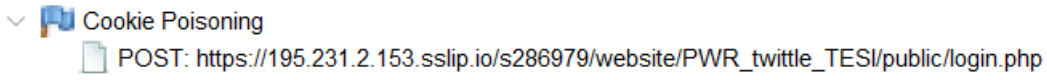


Immagine 4.5: *Avviso di possibile cookie poisoning*

Nel corso della scansione post-mitigazione, OWASP ZAP ha rilevato un potenziale rischio di *cookie poisoning* associato alla pagina `login.php`. L'analisi ha evidenziato che un parametro inviato via POST (user=alessandra) viene successivamente incluso dal server in un cookie denominato last_username, con il valore corrispondente al nome utente inserito.

Sebbene questa pratica sia comune per migliorare l'usabilità, viene considerata potenzialmente rischiosa poiché, in assenza di adeguati controlli, un attaccante potrebbe manipolare il valore del cookie, ad esempio inserendo caratteri speciali come il punto e virgola (;), con l'obiettivo di alterarne la struttura.

Nel caso dell'applicazione Twittle, va evidenziato che il cookie in questione è configurato con i flag HttpOnly, SameSite e Secure che ne limitano l'esposizione e riducono il rischio di manipolazioni lato client. L'avviso è stato pertanto considerato non critico, ma rappresenta un utile promemoria metodologico: in applicazioni più complesse o in contesti più esposti, è preferibile evitare l'inserimento diretto di input utente nei cookie, oppure applicare meccanismi di encoding sicuri.

5.6.3 Potenziale Controllo Utente su Attributi HTML

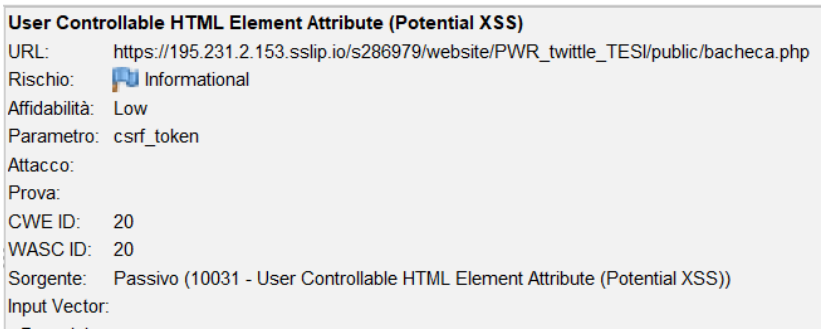


Immagine 4.6: *Avviso di possibile XSS*

- Gravità: Informativo
- Tipo di avviso: Comportamento potenzialmente pericoloso in output HTML

Durante la scansione post-mitigazione, OWASP ZAP ha segnalato la presenza di un attributo HTML il cui valore potrebbe teoricamente essere influenzato da input utente. L'avviso, classificato come "*User Controllable HTML Element Attribute (Potential XSS)*", si basa sulla possibilità che dati non adeguatamente filtrati vengano inseriti all'interno di attributi come `src`, `href` o `value`, creando i presupposti per un potenziale attacco di tipo cross-site scripting (XSS).

Nel caso specifico dell'applicazione Twittle, l'alert è stato generato in corrispondenza del campo nascosto csrf_token presente nel modulo di filtraggio dei post. Tuttavia, il valore di questo campo è correttamente generato lato server e sottoposto a encoding con la funzione `htmlspecialchars()` utilizzando i flag `ENT_QUOTES | ENT_SUBSTITUTE` e codifica `UTF-8`, come mostrato nel seguente frammento di codice:

```
<input type="hidden" name="csrf_token" value="<?php echo
htmlspecialchars($_SESSION['csrf_token'], ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8');
?>">
```

Di conseguenza, è possibile che l'attribuzione dell'alert da parte di ZAP si sia basata esclusivamente su una verifica sintattica e non su un'effettiva vulnerabilità. L'assenza di payload eseguibili o comportamenti anomali durante la scansione conferma che il meccanismo di protezione è stato implementato correttamente

5.6.4 Identificazione di Richieste di Autenticazione

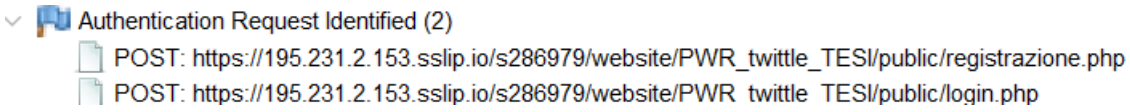


Immagine 4.7: *Avviso di identificazione di richieste di autenticazione*

- Gravità: Informativo
- Tipo di avviso: Pattern riconosciuto come autenticazione

Nel corso dell'analisi post-mitigazione, OWASP ZAP ha rilevato due richieste HTTP classificate come operazioni di autenticazione, relative alle pagine `login.php` e `registrazione.php`. L'avviso, etichettato come *Authentication Request Identified*, non segnala vulnerabilità, ma si limita a riconoscere automaticamente richieste che includono parametri comunemente associati al processo di login, come username, password e csrf_token.

Nello specifico, ZAP ha individuato una richiesta POST verso `registrazione.php`, contenente i parametri username=… e password=..., e una richiesta POST verso `login.php` con i parametri user=… e pwd=....

Ai fini dell'analisi, l'avviso ha un valore puramente descrittivo e conferma che lo strumento ha correttamente identificato i meccanismi di accesso all'applicazione, senza evidenziare criticità tecniche o di sicurezza.

5.7 Confronto tra le Scansioni Pre e Post-Mitigazione

5.7.1 Analisi Quantitativa dei Risultati

L'analisi quantitativa rivela una riduzione significativa del numero complessivo di vulnerabilità identificate. La scansione iniziale aveva individuato 19 vulnerabilità distribuite secondo la seguente classificazione di rischio:

- Vulnerabilità ad alto rischio: 2 istanze (Cross-Site Scripting Persistente e Riflesso)
- Vulnerabilità a medio rischio: 4 istanze (assenza di token anti-CSRF, mancanza di CSP Header, directory browsing, assenza di Anti-clickjacking Header)
- Vulnerabilità a basso rischio: 7 istanze (problematiche di configurazione dei cookie, esposizione di informazioni del server, assenza di header di sicurezza)
- Informazioni di carattere generale: 6 istanze (authentication request, cookie poisoning, suspicious comments)

La scansione post-mitigazione evidenzia un quadro sostanzialmente migliorato, con la completa eliminazione delle vulnerabilità classificate come ad alto e medio rischio:

- Vulnerabilità ad alto rischio: 0 istanze (riduzione del 100%)
- Vulnerabilità a medio rischio: 0 istanze (riduzione del 100%)
- Vulnerabilità a basso rischio: 4 istanze
- Informazioni di carattere generale: 6 istanze (mantenute)

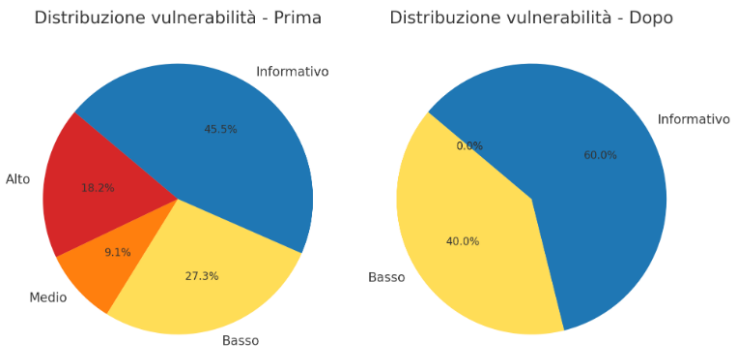


Immagine 4.8: *Distribuzione delle vulnerabilità prima e dopo*

Questo risultato indica una riduzione totale delle vulnerabilità critiche e un miglioramento sostanziale del profilo di sicurezza complessivo dell'applicazione.

5.7.2 Analisi Qualitativa degli Interventi di Mitigazione

Analizzando qualitativamente i dati dei due report, risulta evidente l'efficacia degli interventi mirati alle criticità ad alto e medio rischio.

Cross-Site Scripting (XSS)

L'implementazione di tecniche avanzate di sanitizzazione dell'input utente, unitamente all'adozione di meccanismi di filtraggio e output encoding, ha consentito l'eliminazione completa di entrambe le tipologie di vulnerabilità XSS precedentemente identificate. Le vulnerabilità critiche persistenti e riflesse, inizialmente rilevate, risultano completamente assenti nella seconda scansione. Tale risultato assume particolare rilevanza considerando che le vulnerabilità XSS rappresentano una delle tipologie di exploit più frequentemente sfruttate in ambito applicativo.

Cross-Site Request Forgery (CSRF)

L'integrazione sistematica di token anti-CSRF in tutti i form dell'applicazione e la verifica delle intestazioni Origin e Referer ha determinato l'eliminazione integrale di questa categoria di vulnerabilità. Mentre la scansione iniziale aveva identificato sette istanze di CSRF, evidenziando una significativa esposizione dell'applicazione ad attacchi di tipo request forgery, la scansione successiva non registra alcuna criticità correlata, dimostrando l'efficacia dell'implementazione dei meccanismi di protezione.

Content Security Policy (CSP) e Header di Sicurezza HTTP

L'implementazione di una Content Security Policy rigorosa e di header protettivi quali X-Frame-Options ha permesso l'eliminazione completa delle vulnerabilità associate. Le nove istanze di assenza del CSP e le nove istanze di mancanza degli header anti-clickjacking, inizialmente rilevate da OWASP ZAP, risultano completamente risolte nella scansione post-mitigazione, attestando l'efficacia della configurazione degli header di sicurezza.

Directory Browsing

La vulnerabilità relativa al directory browsing, precedentemente identificata su quattro istanze, è stata interamente risolta. La scansione post-mitigazione non evidenzia alcuna traccia di navigabilità impropria delle directory del sistema.

5.7.3 Vulnerabilità Residuali

Nonostante i significativi progressi documentati, alcune vulnerabilità di basso rischio permangono nella scansione post-mitigazione. Benché non costituiscano rischi immediati per la sicurezza applicativa, potrebbero rappresentare potenziali vettori secondari per attività di *information gathering* da parte di attaccanti.

Esposizione di Informazioni del Server

L'applicazione continua a restituire informazioni tecniche relative ai sistemi utilizzati (PHP e Nginx) attraverso gli header "X-Powered-By" e "Server". Sebbene questa criticità non rappresenti un rischio diretto immediato, potrebbe facilitare attività di ricognizione, incrementando potenzialmente il rischio di attacchi mirati e sofisticati.

Header di Sicurezza Aggiuntivi

Gli header Strict-Transport-Security e X-Content-Type-Options risultano ancora non configurati nelle risorse statiche, in particolare ai fogli di stile (.css) e agli script JavaScript (.js), nonostante fossero stati considerati nella fase di mitigazione. Pur essendo classificati come vulnerabilità minori, rappresentano opportunità non sfruttate per il rafforzamento della sicurezza applicativa, particolarmente per la protezione contro attacchi man-in-the-middle e MIME-sniffing.

5.7.4 Considerazioni Critiche

L'analisi comparativa dei risultati dimostra che le strategie implementate hanno prodotto un miglioramento significativo e misurabile della sicurezza dell'applicazione Twittle. L'eliminazione completa delle vulnerabilità più critiche attesta la validità tecnica e metodologica delle misure correttive adottate.

Tuttavia, la persistenza di alcune criticità minori sottolinea la natura iterativa e continua del processo di miglioramento della sicurezza. Tale processo deve essere concepito come un'attività ciclica, orientata al perfezionamento incrementale della configurazione e della gestione applicativa. Un successivo ciclo di mitigazione potrebbe concentrarsi sugli aspetti informativi e sulle vulnerabilità residuali per consolidare ulteriormente la robustezza dell'applicazione.

La scansione post-mitigazione, pertanto, non rappresenta esclusivamente la conclusione di un ciclo di interventi correttivi, ma costituisce il punto di partenza per successive attività volte a mantenere e consolidare un elevato livello di sicurezza nel lungo termine, in conformità alle best practice delineate dagli standard OWASP e dalla comunità internazionale di sicurezza informatica.

Capitolo 6

Conclusione

La sicurezza delle applicazioni web rappresenta oggi una componente fondamentale all'interno dello sviluppo software, non solo in ambito aziendale ma anche in contesti formativi. Il lavoro svolto in questa tesi ha voluto affrontare in modo pratico e sistematico il tema della gestione delle vulnerabilità applicative, utilizzando come caso di studio il progetto Twittle, una piattaforma web di microblogging progettata a scopo didattico.

Il percorso intrapreso si è sviluppato attraverso un processo strutturato, articolato in tre fasi principali: l'analisi iniziale delle vulnerabilità, la successiva implementazione di contromisure mirate e, infine, la verifica post-mitigazione. La prima fase ha permesso di individuare numerose criticità all'interno dell'applicazione, tra cui vulnerabilità ben note come il Cross-Site Scripting, nelle sue varianti persistente e riflessa, l'assenza di protezioni contro attacchi CSRF, la mancanza di configurazioni di sicurezza a livello di header HTTP, e altre problematiche di tipo informativo e configurativo. Tali risultati hanno fornito una base solida per orientare le azioni correttive.

La seconda fase ha riguardato l'intervento puntuale su ciascuna delle vulnerabilità rilevate. Attraverso tecniche consolidate come la sanitizzazione e l'encoding dell'output, l'introduzione di token anti-CSRF, la definizione di politiche CSP, la gestione sicura delle sessioni e l'impostazione corretta di cookie e header HTTP, è stato possibile intervenire in modo mirato sulle debolezze del sistema. L'intervento è stato condotto tenendo conto sia delle best practice suggerite dalla comunità di sicurezza web, sia delle specificità dell'ambiente applicativo.

Il momento di verifica finale ha rappresentato la fase conclusiva e più significativa del processo. Attraverso una nuova scansione effettuata con OWASP ZAP, è stato possibile valutare con precisione l'efficacia delle misure implementate. I risultati ottenuti hanno confermato in maniera netta la bontà dell'approccio adottato: tutte le vulnerabilità classificate come ad alto e medio rischio sono state completamente eliminate. Alcune segnalazioni minori, di livello basso o puramente informativo, sono rimaste presenti anche dopo l'implementazione delle contromisure. In particolare, si tratta per lo più di situazioni non immediatamente sfruttabili da un attaccante, ma che indicano un margine di miglioramento residuo nella configurazione della sicurezza.

Questi risultati positivi non devono tuttavia essere interpretati come una condizione di sicurezza definitiva. La sicurezza applicativa, per sua natura, è un processo continuo e in evoluzione. Ogni modifica al codice, ogni aggiornamento o cambiamento infrastrutturale, può potenzialmente introdurre nuove vulnerabilità. È quindi necessario adottare un approccio ciclico, che preveda non solo la risoluzione dei problemi noti, ma anche il monitoraggio costante e la ripetizione periodica delle attività di analisi e verifica.

Ragionando più in generale, l'esperienza maturata nel corso di questa tesi consente di riflettere sullo schema metodologico adottato. L'approccio seguito, basato sull'identificazione sistematica delle vulnerabilità, la loro mitigazione mirata e la successiva validazione tecnica, si è dimostrato efficace sotto diversi punti di vista. In primo luogo, ha offerto una struttura chiara e replicabile, capace di guidare l'intervento su problemi concreti in modo ordinato e documentabile. Inoltre, l'utilizzo di strumenti automatizzati come OWASP ZAP ha reso il processo accessibile anche in un contesto a risorse limitate, permettendo un'analisi rapida e ripetibile.

Tuttavia, non si possono ignorare alcune debolezze che questo schema metodologico comporta, soprattutto quando lo si applica in contesti reali. Una prima criticità riguarda la dipendenza dagli strumenti di scansione automatica, che, per quanto avanzati, non possono sostituire del tutto l'analisi manuale o l'esperienza di un penetration tester. Vulnerabilità logiche o errori di configurazione complessi possono sfuggire a una scansione automatizzata.

È proprio da queste osservazioni che emerge una consapevolezza importante: lo schema metodologico adottato in questo lavoro funziona e produce risultati concreti, ma per essere efficace nel lungo periodo deve essere inserito in una visione più ampia e integrata. Deve diventare parte di una cultura della sicurezza che coinvolga tutte le fasi del ciclo di vita del software e tutte le figure coinvolte, dallo sviluppatore al sistemista, fino al responsabile della sicurezza. Solo in questo modo la sicurezza può diventare un processo continuo, e non una semplice attività correttiva da attuare una tantum.

In conclusione, l'esperienza sviluppata attraverso il progetto Twittle dimostra che è possibile migliorare significativamente la sicurezza di una web application attraverso un approccio pratico, iterativo e strutturato. I risultati ottenuti sono incoraggianti e suggeriscono che la metodologia adottata possa essere applicata con successo anche in contesti più complessi.

Il progetto si chiude dunque con la consapevolezza di aver costruito un impianto metodologico solido, replicabile e tecnicamente valido, capace di affrontare le principali sfide della sicurezza applicativa in modo concreto e consapevole.

Ringraziamenti

In primis, desidero ringraziare il mio relatore, il professor Andrea Atzeni, per la disponibilità, l'attenzione e la guida che ha dato, con infinita pazienza, in ogni step della realizzazione dell'elaborato e per i preziosi consigli che han contribuito alla sua stesura.

Un ringraziamento speciale va alla mia famiglia: mamma, papà e Andri. Senza il vostro sostegno e la fiducia che mi avete sempre dimostrato, non so se oggi sarei qui a scrivere queste righe. A mia madre rivolgo un pensiero particolare: sei stata e continui a essere la mia più grande ispirazione. Con la tua forza, la tua dolcezza e la tua testardaggine mi hai insegnato cosa significa affrontare la vita con coraggio e dignità. Ogni mio traguardo porta con sé un po' di te.

Un grazie di cuore va ai miei amici del liceo, Elisa, Amedeo e Aurora. Ho chiuso gli occhi un attimo e sono passati dieci anni dal nostro primo incontro. Siete la cosa più bella che il D'Aze mi abbia lasciato. E come Sex and The City, voi siete le mie anime gemelle col quale spero di continuare a fare cenette romantiche il resto della mia vita.

Un ringraziamento affettuoso va alla mia cara amica Ilaria, la mia compagna di vita. Nonostante gli oltre mille chilometri che ci separano, non hai mai smesso di starmi accanto, sia nei momenti di gioia che in quelli più difficili. La distanza non ha fatto che rafforzare il nostro legame, e mi auguro che, anche se non sempre fisicamente, continuerai a essere parte della mia vita in ogni traguardo e in ogni nuova avventura.

Un ringraziamento speciale va al mio amico e partner cunese, Savino. Non so se avrei raggiunto gli stessi risultati quest'anno senza il tuo supporto costante e la tua presenza, hai saputo ascoltarmi, spronarmi e strappare un sorriso quando serviva di più. Uno degli eventi più rilassanti e piacevoli delle mie settimane rimane passare i pomeriggi a Moncalieri e cucinare assieme, oltre che batterti a ping pong. Sei scarso.

Un grazie sincero anche ad Antonio, Elisa e Peter: sebbene la nostra amicizia sia nata solo recentemente, ha saputo diventare qualcosa di importante. Spero che il tempo ci regali molte altre occasioni per rafforzare questo legame. Rimanete comunque scarsi a ping pong. Soprattutto te Elisa.

Infine, voglio ringraziare tutti gli amici che mi sono stati vicini sin dai primi anni di università: Giuseppe, Yahya, Mattia, Riccardo, Giorgio, Gaia, Arianna, e tutti quelli che ho conosciuto lungo il percorso. Grazie per le risate, per i momenti leggeri e per aver trasformato anche le giornate più impegnative in ricordi piacevoli. A voi, compagni di studio e di merenda, devo gran parte della serenità che mi ha accompagnata in questi anni.

Bibliografia

[1] SentinelOne, *Key Cyber Security Statistics for 2025*. URL: https://www.sentinelone.com/cybersecurity-101/cybersecurity/cyber-security-statistics/#:~:text=Cost%20and%20Frequency%20of%20Cyber%20Attacks,-The%20frequency%20of&text=The%20reported%20costs%20of%20cyber,at%20%2423%20trillion%20by%202027.

[2] Security Summit, *Rapporto Clusit sulla Cybersecurity in Italia e nel Mondo 2025*. URL: https://clusit.it/rapporto-clusit/

[3] OWASP Foundation, *Software Assurance Maturity Model (SAMM). URL: https://owasp.org/www-project-samm/*

[4] OWASP Foundation, OWASP Code Review Guide. URL: https://owasp.org/www-project-code-review-guide/

[5] SonarQube | sever, *SonarQube Server 2025.3 Documentation*, URL: https://docs.sonarsource.com/sonarqube-server/latest/ . UCSF Information Technology, *SonarQube - Continuous Inspection of Code Quality and Security*. URL: https://it.ucsf.edu/how-to/sonarqube-continuous-inspection-code-quality-and-security?utm_source=chatgpt.com

[6] ZAP by Checkmarx, ZAP by Checkmarx Desktop User Guide. URL: https://www.zaproxy.org/docs/desktop/

[7] PortSwigger (giugno 3, 2025), *Integrating with CI/CD platforms*. URL: https://portswigger.net/burp/documentation/dast/user-guide/ci-cd?utm_source=chatgpt.com

[8] Sucuri Security Blog, The Definitive WordPress Security Guide. URL: https://sucuri.net/guides/wordpress-security/

[9] Rana Khalil, ZAP Tutorial - How to Set Up ZAP to Work with Browser, URL: https://rkhal101.github.io/_posts/WAVS/ZAP/zap_browser_setup#:~:text=the%20web%20application.-,Configure%20ZAP%20as%20proxy,our%20next%20and%20final%20section.

Devon, OWASP ZAP for Dummies. URL: https://devonblog.com/security/owasp-zap-for-dummies/

[10] ZAP by Checkmarx, *Scan Policy*. URL: https://www.zaproxy.org/docs/desktop/start/features/scanpolicy/

[11] Apache Software Foundation, *Apache HTTP Server - .htaccess Configuration*. URL: https://httpd.apache.org/docs/current/howto/htaccess.html.

[12] Mozilla Developer Network, *Content Security Policy (CSP)*. URL: https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP.

[13] OWASP Foundation, *Session Management Cheat Sheet. URL: https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html*.

[14] Mozilla Developer Network (MDN), *Secure Cookie Attributes. URL: https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Cookies*.

[15] OWASP Cheat Sheet Series, *Cross-Site Request Forgery (CSRF) Prevention Cheat Sheet*. URL: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html.

[16] OWASP; *DOMPurify*. URL: https://github.com/cure53/DOMPurify.

[17] OWASP Cheat Sheet Series, *Cross-Site Request Forgery (CSRF) Prevention Cheat Sheet. URL: https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html*.

[18] OWASP Cheat Sheet Series, *PHP Configuration Cheat Sheet*. URL: https://cheatsheetseries.owasp.org/cheatsheets/PHP_Configuration_Cheat_Sheet.html.

[19] OWASP Cheat Sheet Series, User Privacy Protection Cheat Sheet. URL: https://cheatsheetseries.owasp.org/cheatsheets/User_Privacy_Protection_Cheat_Sheet.html.

[20] MDN Web Docs, *Strict-Transport-Security (HSTS). URL: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Strict-Transport-Security*.

[21] OWASP Foundation, OWASP Code Review Guide. URL: https://owasp.org/www-project-code-review-guide/ .

Appendice

[i] – Report della Scansione Pre-Mitigazione

In questa sezione viene riportato il report completo generato dallo strumento **OWASP ZAP** durante la fase iniziale di valutazione dell'applicazione *Twittle*, prima dell'introduzione delle contromisure di sicurezza descritte nel Capitolo 4.
Il report include l'elenco dettagliato delle vulnerabilità rilevate, suddivise per livello di rischio (alto, medio, basso, informativo), e descrive per ciascuna vulnerabilità le informazioni tecniche, i parametri coinvolti, i metodi HTTP, gli URL affetti e i riferimenti CWE.

Il report completo è allegato in formato HTML alla tesi come file "ZAP-Report3-.html".

[ii] – Report della Scansione Post-Mitigazione

Questo documento presenta il risultato della seconda scansione OWASP ZAP, eseguita dopo l'applicazione delle contromisure di sicurezza. L'obiettivo è stato verificare l'efficacia degli interventi correttivi e la scomparsa delle vulnerabilità precedentemente individuate.

Il report evidenzia una netta riduzione del numero e del livello di rischio delle vulnerabilità, confermando l'efficacia della mitigazione implementata nel Capitolo 4.

Il report completo è allegato in formato HTML alla tesi come file "ZAP-Report-post.html".